

# 1. REQUIREMENTS / SCOPE

## 1.1 Project Overview

**Thrill Mill Club** is a sports-club community management and Turf booking application designed to help the club manage Teams, memberships, shared credits, Turf bookings and member participation digitally.

The MVP focuses on **Turf operations and Team-based community management**.

The initial MVP does not include Pickleball, multiple-sport management or advanced competitive match verification.

The application is centered around **Teams**.

A Team is a group of registered Thrill Mill Club members who participate in Turf activities together.

Each Team has:

- A Host
- An optional Co-host
- Members
- A membership plan
- A shared credit wallet
- Turf bookings
- Booking participants
- Activity history
- Usage information
- Team chat

A member may belong to multiple Teams, with a different role in each Team.

The MVP provides the core functionality required to manage the complete journey:

User Registration / Login



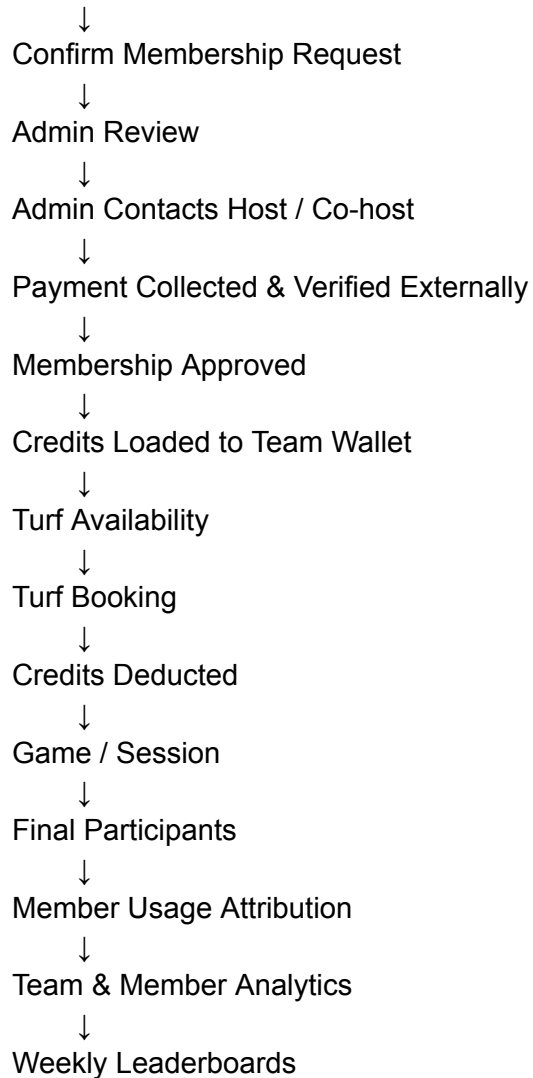
Create / Join Team



Add Team Members



Select Membership Plan



The core financial principle remains:

**The Team is the financial unit; the individual member is the participation and analytics unit.**

This is consistent with the existing blueprint's Team/Network-owned credit model, where credits belong to the group rather than individual members.

---

## 1.2 MVP Objective

The primary objective of the MVP is to validate whether **Thrill Mill Club can operate its Team community and Turf booking process digitally**, while reducing manual coordination around memberships, Turf reservations, credit usage and Team activity.

The MVP should enable the club to move from:

Manual / WhatsApp Coordination



Team Creation



Membership Request



Admin Verification



Team Credit Wallet



Turf Booking



Participation



Usage & Activity

The MVP is intended to be a **functional production-oriented application**, not merely a visual prototype.

Because the initial development window is limited, the implementation should prioritize **correct financial, credit, booking and authorization logic** over advanced features. The original blueprint similarly identifies wallet, booking, cancellation and usage attribution as the highest-risk chain.

---

## 1.3 Primary User Scope

The MVP has four operational roles:

### Member

A registered Thrill Mill Club user who can participate in one or more Teams.

A Member can:

- Create or join eligible Teams
- View Team information
- View Team wallet balance
- View Team activity
- View their completed participation
- View credits used through completed participation
- Participate in Team chat

- Receive notifications
- View booking history
- View weekly leaderboards

## **Host**

The primary manager of a Team.

The Host can additionally:

- Manage Team members
- Assign a Co-host
- Initiate Turf bookings
- Select booking participants
- Manage Team booking activities
- View Team wallet information
- View Team usage and activity
- Cancel eligible bookings

## **Co-host**

A secondary Team manager.

The Co-host can perform the operational actions assigned to the Team management role, including:

- Initiating bookings
- Selecting participants
- Managing booking activities
- Viewing Team wallet information
- Managing permitted Team operations
- Cancelling eligible bookings

The exact Host/Co-host permission difference should remain configurable.

## **Admin**

The Thrill Mill Club platform operator.

The Admin has platform-level access and can:

- View all Teams
- Create Teams on behalf of customers
- View Team members
- View Host and Co-host contact information

- Review membership requests
- Contact Host / Co-host regarding payment
- Verify externally collected membership payment
- Approve membership
- Load credits into a Team wallet
- View Team wallet and ledger information
- View all bookings
- Create bookings on behalf of Teams
- Manage Turf availability
- Block/unblock Turf slots
- View operational activity and basic analytics
- Perform authorized administrative adjustments

The Admin operates through the **same Thrill Mill Club mobile application**, with additional capabilities exposed through role-based authorization. The original blueprint explicitly corrected the architecture to one mobile application with role-based capabilities rather than a separate Admin application.

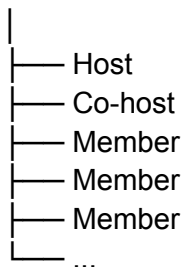
---

## 1.4 Team Scope

A **Team** is the central community and operating unit of the MVP.

Example:

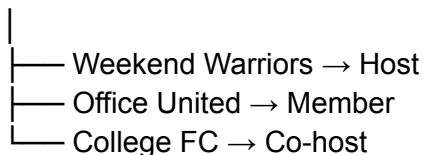
Weekend Warriors



A Team may contain multiple registered members.

A member may belong to multiple Teams:

Member



Each Team maintains its own independent:

- Members
- Membership
- Wallet
- Credits
- Bookings
- Participation records
- Chat
- Activity
- Usage data

Credits belonging to one Team must never be mixed with another Team. This independence is already established in the original blueprint's Network model.

---

## 1.5 Team Wallet Scope

Credits belong to the **Team**, not to individual members.

The core relationship is:



There are no individual member credit wallets and no member-to-member credit transfers in the MVP.

Members can view:

- Team wallet balance
- Booking-related credit consumption
- Their own credits used through completed participation
- Relevant Team activity

The Host and Co-host operate the Team's booking-related activities, while the Admin controls membership approval and administrative credit loading.

This preserves the financial model already established in the blueprint.

---

## 1.6 Membership Plan Scope

The MVP contains two Team membership plans.

### ₹10,000 Membership

- Membership Turf price: **₹800/hour**
- Standard Turf price: **₹1,000/hour**
- Maximum **3 playing hours at membership pricing**
- The limit uses a **rolling 24-hour window**
- The limit is based on **playing hours**, not number of bookings, calendar day or number of members

Example:

2-hour booking

+

1-hour booking

=

3 membership-priced hours

Any additional eligible playing hours within the applicable rolling 24-hour window use the standard Turf price.

### ₹25,000 Membership

- Membership Turf price: **₹800/hour**
- No 3-hour rolling-24-hour restriction

Credits do not expire under the current MVP business rule.

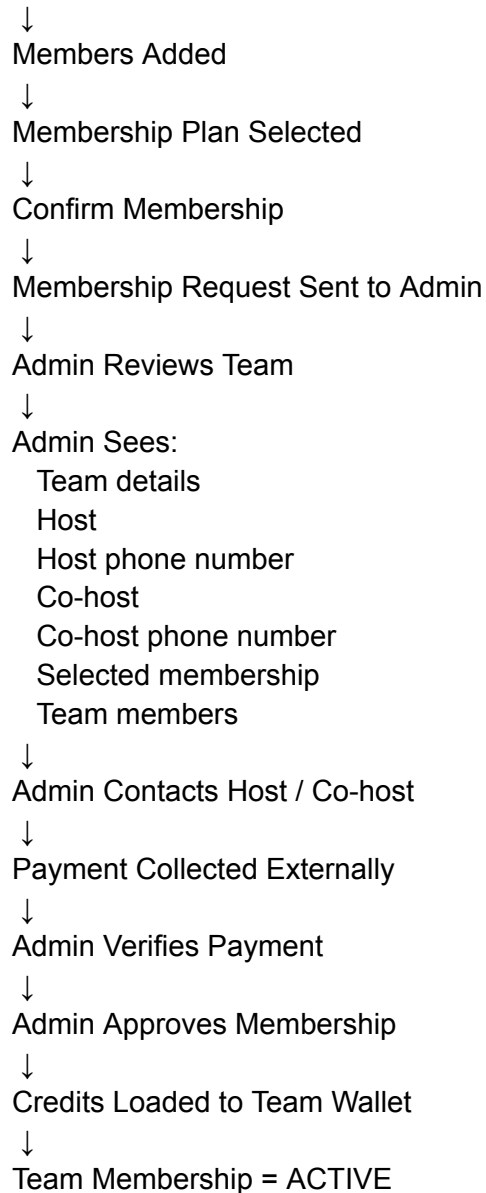
---

## 1.7 Membership Request & Admin Approval Scope

This is a **newly updated part of the MVP** and replaces the old online payment flow.

The membership process is:

Team Created



The application itself does **not** process the membership payment through Razorpay or another payment gateway in the MVP.

The Admin is responsible for the external payment verification and activation process.

The Host/Co-host contact information is displayed to the Admin specifically to allow efficient communication regarding:

- Payment collection
- Membership confirmation
- Team details
- Booking-related coordination



This contact information is **Admin-only operational information** and must not be exposed unnecessarily to other users.

---

## 1.8 Turf Booking Scope

The MVP supports:

- Viewing Turf availability
- Selecting date
- Selecting available slots
- Selecting duration
- Calculating applicable price
- Checking Team wallet balance
- Selecting participants
- Holding a slot for 1 minute
- Confirming a booking
- Deducting Team credits
- Viewing booking history
- Booking notifications

The booking engine must apply the current membership pricing rules and Team wallet rules. The original blueprint establishes the one-minute hold, backend availability checks and atomic booking/credit deduction requirements.

The MVP also supports **Admin-assisted booking**, where an Admin can use a Team ID to create a Turf booking on behalf of that Team.

---

## 1.9 Participant & Usage Scope

For each booking, the Host, Co-host or authorized Admin may select the participating Team members.

The final participant list is recorded against the individual booking.

After the session is completed:

Actual booking credits consumed

÷

Final participant count

=

Credits Used Through Completed Participation

Example:

Booking cost = 1,800 credits

Participants = 6

$1,800 \div 6$

=

300 credits/member

This is an **analytics attribution**, not a transfer of credits to individual members.

---

## 1.10 Cancellation & Refund Scope

The current MVP cancellation rule remains:

Cancellation  $\geq$  24 hours before session

↓

Full Team credit refund

↓

Applicable discounted-hour usage restored

Cancellation < 24 hours before session

↓

No credit refund

The cancellation decision must be determined by backend/server time rather than the mobile application's local clock.

---

## 1.11 Leaderboard Scope

The MVP contains weekly leaderboards based on actual completed activity.

### Weekly Team Leaderboard

Ranking metric:

**Number of completed Turf bookings**

### Weekly Member Leaderboard

Ranking metric:

### **Credits Used Through Completed Participation**

The leaderboard runs from Monday 00:00 through Sunday 23:59 using the configured business timezone.

Verified match results, scores and tournament ranking are outside the MVP.

---

## **1.12 Explicitly Outside MVP**

The following remain outside the initial MVP:

- Pickleball booking
- Multiple sports management
- Verified match results
- Score-based ranking
- Tournament management
- Advanced AI moderation
- Individual wallet-to-wallet transfers
- Individual payment splitting
- Automatic participant credit collection
- Advanced financial settlement
- Complex enterprise Admin dashboards
- Advanced reporting
- Media-heavy chat
- Public iOS release during the Android-first MVP

These exclusions are consistent with the original scope, except that the old Razorpay-based payment implementation is now replaced by the new Admin-verified external payment process.

---

## **1.13 MVP Success Criteria**

The MVP is successful when a real Thrill Mill Club user can:

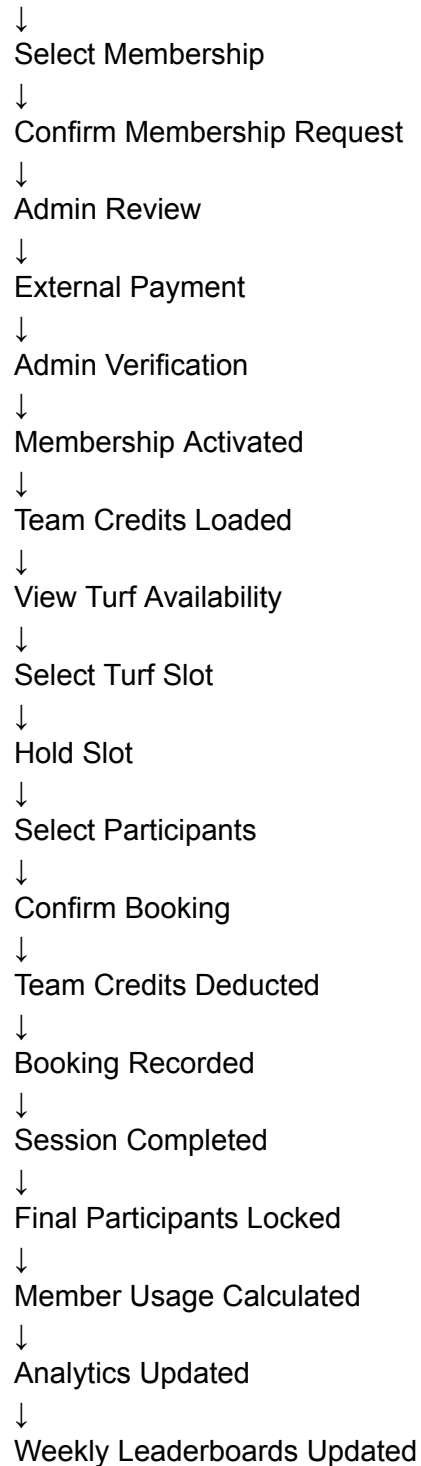
Register / Login

↓

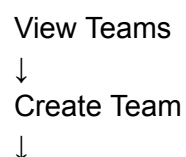
Create or Join Team

↓

Add / Accept Team Members



The Admin must additionally be able to:



Review Membership Request



Contact Host / Co-host



Verify Payment



Activate Membership



Load Team Credits



Create Booking for Team



Manage Turf Availability

This becomes the **baseline scope** for the rest of the new document.

## 2. USER ROLES + PERMISSIONS

### 2.1 Role Architecture

Thrill Mill Club uses a **two-level role model**.

#### Platform-Level Role

A user has one platform-level role:

- **MEMBER**
- **ADMIN**

The Platform Role defines the user's access across the Thrill Mill Club application.

#### Team-Level Role

Within each Team, a user may have one of:

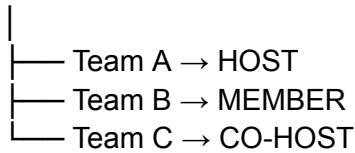
- **HOST**
- **CO-HOST**
- **MEMBER**

Team-level roles are independent of the user's platform-level role.

Therefore, the same person may have different roles across different Teams.

Example:

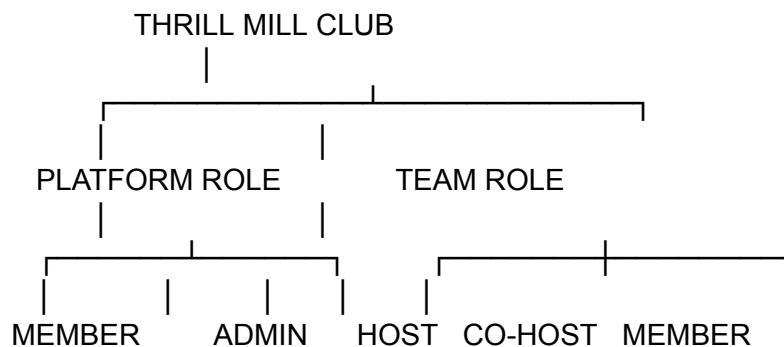
User



This follows the original blueprint's distinction between global platform permissions and Team-specific permissions.

---

## 2.2 Role Hierarchy



The **Admin** is a platform-level operational role and is not simply another Team role.

Host, Co-host and Member permissions apply only within the specific Team in which the user holds that role.

The original blueprint establishes that Admin is global while Host/Co-host/Member are Team-specific.

---

## 2.3 Member

A Member is a registered Thrill Mill Club user who can participate in one or more Teams and Turf activities.

**Member can:**

- Register and log in

- Maintain their profile
- View Teams they belong to
- Join an eligible Team
- Create a Team where permitted
- View Team members
- View Team membership information
- View Team wallet balance
- View relevant Team credit activity
- View Turf availability
- Participate in Turf activities
- View booking history
- View completed participation history
- View their credits used through completed participation
- Access Team chat
- Receive notifications
- View weekly Team leaderboard
- View weekly Member leaderboard
- View personal activity and usage insights

The underlying capabilities are consistent with the original Member scope.

#### **Member cannot:**

- Directly modify the Team wallet
- Directly add or remove Team credits
- Transfer credits to another Member
- Perform Admin operations
- Verify or approve membership payments
- Activate Team membership
- Block or unblock Turf slots
- Modify Admin-controlled operational records

---

## **2.4 Host**

A Host is a Member who has management authority within a specific Team.

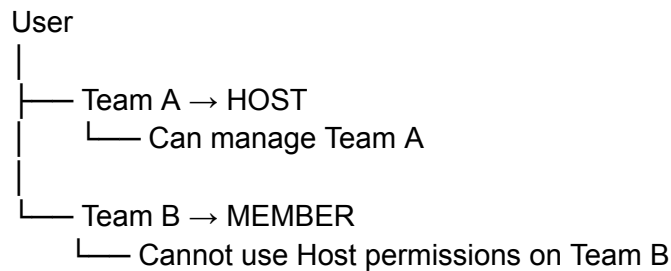
#### **Host can:**

- View Team information
- Manage Team members
- Add eligible registered Members

- Assign or change Co-host where authorized
- Initiate Turf bookings
- Select booking participants
- Modify participants before the permitted cutoff
- Cancel eligible bookings
- View Team wallet information
- View Team wallet activity
- View Team bookings
- View Team usage information
- Access and manage Team chat according to permissions

The Host's authority applies **only to the Team where the user is Host**.

For example:



This Team-specific permission model is explicitly defined in the original blueprint.

---

## 2.5 Co-host

A Co-host is a Team Member who receives secondary management authority.

### Co-host can:

- View Team information
- Manage permitted Team activities
- Initiate Turf bookings
- Select booking participants
- Modify participants before the permitted cutoff
- Cancel eligible bookings
- View Team wallet information
- View Team bookings
- Access Team management capabilities assigned to the Co-host
- Participate in Team chat moderation where authorized



The exact difference between Host and Co-host should remain limited to clearly defined permissions and should not create unnecessary complexity in the MVP. The original blueprint similarly leaves the precise Host/Co-host distinction configurable.

---

## 2.6 Admin

The Admin is the **platform-level Thrill Mill Club operator**.

Admin access is global and does not depend on being an ordinary member of a Team.

The Admin uses the **same Thrill Mill Club mobile application**, but role-based authorization exposes additional Admin capabilities.

### Admin can:

#### Team Management

- View all Teams
- Search Teams
- Search by Team ID
- View Team members
- View Host and Co-host
- View Host phone number
- View Co-host phone number
- View Team activity
- Create a Team on behalf of customers
- Add existing registered Members
- Assign Host
- Assign Co-host

#### Membership Management

- View membership requests
- View selected membership plan
- Review Team details
- Contact Host or Co-host regarding payment
- Collect/coordinate external payment
- Verify that payment has been received
- Approve membership
- Activate the membership

- Load the corresponding credits into the Team wallet

The Host/Co-host contact details are exposed to the Admin specifically to support operational communication during membership review and payment collection.

## **Booking Management**

- View all bookings
- View booking details
- View participants
- View booking status
- Create a Turf booking on behalf of a Team
- Search for a Team using Team ID
- Select Turf/date/time
- Apply the same booking rules used by Host/Co-host
- Cancel bookings according to authorized policy

## **Turf Management**

- View Turf availability
- Block Turf slots
- Unblock Turf slots
- Manage Turf availability for maintenance, private events, closure or other operational reasons

## **Credit & Wallet Operations**

- View Team wallet
- View wallet transaction history
- Load initial membership credits
- Perform authorized manual credit adjustments
- Provide an adjustment reason
- Trigger an auditable credit operation

## **Monitoring**

- View Team activity
- View booking activity
- View member usage information
- View basic operational analytics
- View weekly leaderboards
- Receive operational booking notifications

The original blueprint already gives the Admin global visibility over Teams, bookings, Turf and wallet operations; the updated MVP additionally makes Team creation, membership activation and Admin-assisted booking explicit.

---

## 2.7 Admin Membership Review Permissions

Membership approval is a specific Admin responsibility.

The workflow is:

Team  
↓  
Members Added  
↓  
Membership Selected  
↓  
Confirm Membership  
↓  
Request Sent to Admin  
↓  
Admin Reviews Request

During review, the Admin must be able to see:

Team Name  
Team ID  
Selected Membership Plan  
Team Members  
Host Name  
Host Phone Number  
Co-host Name  
Co-host Phone Number

The Admin can then:

Contact Host / Co-host  
↓  
Collect / Verify External Payment  
↓  
Approve Membership

↓  
Load Credits  
↓  
Team = ACTIVE

Phone numbers are **operational Admin information** and should not be exposed to Members merely because they belong to the same Team.

---

## 2.8 Admin-Assisted Team Creation

The Admin can create a Team on behalf of customers.

The Admin can:

Create Team  
↓  
Enter Team Details  
↓  
Add Existing Members  
↓  
Assign Host  
↓  
Assign Co-host  
↓  
Select Membership  
↓  
Review  
↓  
Create Team

After creation:

Team ID Generated  
↓  
Membership = PAYMENT\_PENDING  
↓  
Admin Contacts Host / Co-host  
↓  
External Payment  
↓  
Verification  
↓

Activation



Credits Loaded

The resulting Team is **not a special Admin-owned Team**. Once created, it follows the same Team structure and business rules as any other Team.

---

## 2.9 Admin-Assisted Booking Permission

The Admin may create a booking on behalf of an active Team.

The Admin workflow is:

Admin



Create Booking



Enter Team ID



Team Found



Verify Team / Membership



Select Turf



Select Date / Time



System Calculates Price



Check Team Wallet



Confirm Booking



Team Credits Deducted

The Admin should use the **same booking engine and pricing logic** as Host/Co-host booking.

The Admin does not manually calculate or directly modify the credit deduction.

The backend remains authoritative for availability, pricing, wallet balance, booking confirmation and ledger creation.

---

## 2.10 Role Isolation Rules

### **Rule 1 — Admin is global**

ADMIN ≠ TEAM ROLE

An Admin's access comes from the platform-level role.

### **Rule 2 — Team roles are contextual**

A Host in Team A is not automatically a Host in Team B.

### **Rule 3 — Member access is Team-specific**

A Member can only access information belonging to Teams in which they have valid membership.

### **Rule 4 — UI restrictions are not security**

Hiding an Admin button is not sufficient.

Backend authorization must independently verify the user's permissions before executing sensitive operations.

### **Rule 5 — Financial operations are Admin-controlled**

Members cannot directly modify:

- Team wallet balance
- Credit ledger
- Membership activation
- Administrative credit adjustments

### **Rule 6 — Admin actions are auditable**

Sensitive Admin operations must create audit records, including:

- Membership approval
- Credit loading
- Credit adjustment
- Role change
- Booking override

- Turf blocking
- Refund or cancellation override

The original blueprint requires sensitive Admin actions affecting credits or booking history to be auditable.

---

## 2.11 Permission Matrix

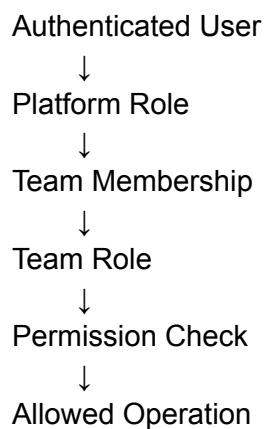
| Capability                        | Member | Host | Co-host | Admin |
|-----------------------------------|--------|------|---------|-------|
| Login / Profile                   | ✓      | ✓    | ✓       | ✓     |
| View own Teams                    | ✓      | ✓    | ✓       | ✓     |
| View Team members                 | ✓      | ✓    | ✓       | ✓     |
| View Team wallet                  | ✓      | ✓    | ✓       | ✓     |
| Create Team                       | ✓*     | —    | —       | ✓     |
| Manage Team members               | ✗      | ✓    | ✓*      | ✓     |
| Assign Co-host                    | ✗      | ✓    | ✗       | ✓     |
| Select Membership                 | ✓*     | ✓    | ✓*      | ✓     |
| Submit Membership Request         | ✓      | ✓    | ✓       | ✓     |
| Verify External Payment           | ✗      | ✗    | ✗       | ✓     |
| Approve Membership                | ✗      | ✗    | ✗       | ✓     |
| Load Team Credits                 | ✗      | ✗    | ✗       | ✓     |
| Initiate Turf Booking             | ✗*     | ✓    | ✓       | ✓     |
| Create Booking for another Team   | ✗      | ✗    | ✗       | ✓     |
| Select Participants               | ✗*     | ✓    | ✓       | ✓     |
| Modify Participants before cutoff | ✗*     | ✓    | ✓       | ✓     |

|                                |         |    |    |   |
|--------------------------------|---------|----|----|---|
| Cancel Eligible Booking        | ✗*      | ✓  | ✓  | ✓ |
| View Booking History           | ✓       | ✓  | ✓  | ✓ |
| View Team Analytics            | Limited | ✓  | ✓  | ✓ |
| Access Team Chat               | ✓       | ✓  | ✓  | ✓ |
| Moderate Team Chat             | ✗       | ✓* | ✓* | ✓ |
| View Weekly Member Leaderboard | ✓       | ✓  | ✓  | ✓ |
| View Weekly Team Leaderboard   | ✓       | ✓  | ✓  | ✓ |
| Block Turf Slot                | ✗       | ✗  | ✗  | ✓ |
| Unblock Turf Slot              | ✗       | ✗  | ✗  | ✓ |
| View All Teams                 | ✗       | ✗  | ✗  | ✓ |
| Platform Analytics             | ✗       | ✗  | ✗  | ✓ |
| Manual Credit Adjustment       | ✗       | ✗  | ✗  | ✓ |

\* Subject to the specific MVP flow/business rule.

## 2.12 Final Role Principle

The application should always determine permissions using:





Therefore:

**One user account can behave as a Member, Host, Co-host or Admin depending on the context in which the application is being used.**

## 3. BUSINESS RULES

This section defines the rules that determine **how Thrill Mill Club actually operates**. These rules are authoritative for the application, backend logic, database behavior, and testing.

The UI may display these rules, but the backend must enforce all business-critical rules.

---

### 3.1 Core Business Principles

The MVP follows these fundamental principles:

1. **Teams are the primary community unit.**
2. **Credits belong to the Team, not individual Members.**
3. **Membership and Team roles are separate concepts.**
4. **The Admin has platform-level authority.**
5. **Booking price is determined by the Team's active membership and applicable usage rules.**
6. **The backend is the authority for credits, pricing, availability, permissions and booking state.**
7. **Every financial or administrative credit change must be traceable through a ledger/audit record.**
8. **Historical financial and booking records must not be silently overwritten.**

The underlying blueprint establishes the same Team-owned wallet model and emphasizes server-side enforcement of critical financial and booking rules.

---

### 3.2 Team Creation Rules

A Team can be created through two supported paths.

#### **Member-created Team**

An eligible Member can create a Team through the application.

Member  
↓  
Create Team  
↓  
Enter Team Details  
↓  
Add Members  
↓  
Assign Roles  
↓  
Select Membership  
↓  
Confirm

### **Admin-assisted Team**

The Admin can create a Team on behalf of customers.

Admin  
↓  
Create Team  
↓  
Enter Team Details  
↓  
Add Existing Members  
↓  
Assign Host / Co-host  
↓  
Select Membership  
↓  
Review  
↓  
Create Team

The two paths create the same Team type and follow the same subsequent business rules.

The Team creation process must not create a separate "Admin Team" category.

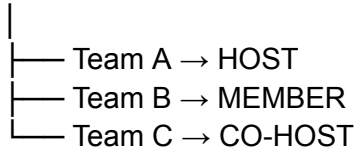
---

## **3.3 Team Membership Rules**

A user can belong to multiple Teams.

Example:

Member



Team membership is independent for every Team.

A user joining one Team does not automatically become a member of another Team.

Only an active Team membership grants normal access to Team resources such as Team chat and Team activity.

---

## 3.4 Team Role Rules

Every Team can have:

- One Host
- Zero or one Co-host
- One or more Members

### Host

The Host has primary Team-management authority.

### Co-host

The Co-host has permitted secondary management authority.

### Member

A normal Team participant has no Team-management authority unless explicitly assigned a management role.

A Host or Co-host's permissions apply only to the Team in which that role exists.

---

## 3.5 Membership Plan Rules

The MVP contains two membership plans.

## ₹10,000 Membership

Normal Turf price:

**₹1,000/hour**

Membership Turf price:

**₹800/hour**

The discounted membership price is available for a maximum of:

**3 playing hours within a rolling 24-hour period.**

The limitation is based on **playing hours**, not:

- Number of bookings
- Calendar day
- Number of members

Example:

2-hour booking

+

1-hour booking

=

3 discounted hours

Any additional playing hour within the applicable rolling 24-hour period is charged at:

**₹1,000/hour**

This rule is already established in the original blueprint.

---

## ₹25,000 Membership

Membership Turf price:

**₹800/hour**

The ₹25,000 plan has **no 3-hour rolling-24-hour restriction** for membership pricing.

---

## 3.6 Credit Expiry Rules

Under the current MVP business rule:

**Team credits do not expire.**

Unused credits remain available in the Team wallet.

Automatic credit expiry must not be implemented unless Thrill Mill Club introduces a future plan-expiry rule.

The blueprint explicitly states that unused credits should not automatically expire under the current model.

---

## 3.7 Membership Request Rules

After a Team has:

- Added the required Members
- Assigned Host/Co-host
- Selected a membership plan

the Team submits the membership request for Admin processing.

Team Setup

↓

Select Membership

↓

Confirm Membership

↓

Membership Request

↓

Admin Review

The application must then show the Team membership as pending activation until the Admin completes the operational process.

---

## 3.8 External Payment Rules

The MVP **does not use Razorpay or an in-app payment gateway** for membership payment.

Payment is collected externally by the Thrill Mill Club Admin.

Possible club-approved payment methods may include:

- UPI
- QR payment
- Bank transfer
- Other approved offline payment methods

The exact payment method is an operational configuration of the club.

The application records the payment for operational and audit purposes but does not directly process the customer's payment.

---

## 3.9 Admin Payment Verification Rules

When reviewing a membership request, the Admin must be able to see:

- Team name
- Team ID
- Selected membership plan
- Team members
- Host name
- Host phone number
- Co-host name
- Co-host phone number

The Admin can use the Host/Co-host contact details to communicate directly regarding:

- Payment
- Membership confirmation
- Team details
- Booking coordination where necessary

The contact details are intended for **Admin operational use** and should not be unnecessarily exposed to other Members.

---

## 3.10 Membership Activation Rules

The activation process is:

Membership Request  
↓  
Admin Reviews  
↓  
Admin Contacts Host / Co-host  
↓  
Payment Collected Externally  
↓  
Admin Verifies Payment  
↓  
Admin Approves Membership  
↓  
Membership = ACTIVE  
↓  
Team Credits Loaded

The Admin must not activate the membership before confirming the payment according to the club's operational process.

The membership activation and initial credit allocation should be performed as a controlled backend operation rather than independent client-side database updates.

---

## 3.11 Initial Credit Allocation Rules

When the Admin successfully activates a membership:

Verified Payment  
↓  
Membership Activation  
↓  
Credit Allocation  
↓  
Team Wallet

The corresponding credit amount is determined by the configured membership plan.

The credit allocation must:

- Belong to the Team
- Create a wallet ledger entry
- Record the operator/Admin
- Record the timestamp
- Reference the associated membership/payment record

A duplicate activation must not allocate the credits twice.

---

## 3.12 Team Wallet Rules

The Team owns the wallet.

Team  
↓  
Team Wallet  
↓  
Credits  
↓  
Turf Booking

Members do not own individual portions of the Team wallet.

There is no:

- Member-to-member credit transfer
- Individual wallet
- Member wallet splitting
- Individual payment splitting

The wallet ledger remains the authoritative record of Team credit activity.

---

## 3.13 Wallet Ledger Rules

Every Team credit change must generate a ledger record.

Supported transaction types include:

MEMBERSHIP\_CREDIT\_ALLOCATION  
BOOKING\_DEBIT



BOOKING\_REFUND  
ADMIN\_ADJUSTMENT  
SYSTEM\_RELEASE

Each ledger entry should record:

- Team
- Transaction type
- Amount
- Balance before
- Balance after
- Reference
- Related booking/payment
- Date/time
- Operator, where applicable

Financial history must not be silently deleted or rewritten. The original blueprint requires each credit balance change to be traceable through a ledger record.

---

## 3.14 Admin Credit Adjustment Rules

If an Admin manually adds or removes Team credits, the operation must require:

- Team
- Amount
- Reason
- Admin identity
- Timestamp
- Ledger reference

Example:

ADMIN\_ADJUSTMENT

Team:

Weekend Warriors

Amount:

+500 Credits

Reason:

Promotional Credit

Operator:  
Admin

Timestamp:  
Recorded automatically

The Admin must not directly overwrite the Team wallet balance.

The adjustment must go through a controlled backend operation and create both a ledger record and an audit record.

---

## 3.15 Turf Availability Rules

A Turf slot can be:

AVAILABLE  
HELD  
CONFIRMED  
BLOCKED  
EXPIRED

A slot is selectable only if it is currently available according to the backend.

Availability must consider:

- Existing confirmed bookings
- Active one-minute holds
- Admin blocks
- Current Turf availability

The original workflow defines these checks before displaying a slot as bookable.

---

## 3.16 One-Minute Hold Rules

When a valid booking attempt begins:

AVAILABLE  
↓

HELD



1-minute timer

The hold expires after one minute if booking confirmation does not occur.

The expiry timestamp is controlled by the backend.

The mobile countdown is only a visual representation and cannot determine whether the hold is still valid.

---

## 3.17 Booking Eligibility Rules

A Turf booking can only proceed when:

- User is authenticated
- User has the required booking permission
- Team exists
- Team membership is valid
- Membership is active where required
- Turf is available
- Slot is not blocked
- Slot hold is valid where applicable
- Team has sufficient credits
- Participants are eligible
- Final calculated price is valid

The same booking rules apply whether the booking is created by:

**Host**

**Co-host**

or

**Admin**

The Admin is simply an additional authorized booking operator.

---

## 3.18 Admin-Assisted Booking Rules

An Admin may create a Turf booking on behalf of a Team.

The Admin can receive a request containing:

- Team ID
- Preferred date
- Preferred time

The Admin can then:

Team ID  
↓  
Find Team  
↓  
Verify Team  
↓  
Select Turf  
↓  
Select Date / Time  
↓  
System Calculates Price  
↓  
Check Team Wallet  
↓  
Confirm Booking  
↓  
Credits Deducted

The Admin does not manually determine the booking price or manually deduct credits.

The same backend booking and wallet logic must be used for Admin-assisted booking.

---

## 3.19 Booking Price Calculation Rules

The final booking price is determined by:

Team  
↓  
Active Membership Plan

↓  
Booking Duration  
↓  
Rolling 24-hour Eligible Usage  
↓  
Applicable Membership Hours  
↓  
Standard-priced Hours  
↓  
Final Booking Cost

For the ₹10,000 plan:

Available discounted hours  
=  
3 – eligible discounted hours already used

The backend must determine the applicable amount.

The mobile application may show an estimated price for user experience, but the backend calculation is authoritative.

---

## 3.20 Mixed-Rate Booking Rules

A single booking may contain both membership-priced and standard-priced hours.

Example:

Hour 1 → ₹800  
Hour 2 → ₹1,000  
  
Total → ₹1,800

The final Team wallet debit is the **total actual booking cost**.

Member usage is not calculated separately by pricing category.

The completed booking charge is divided by the final participant count for participation usage attribution.

---

## 3.21 Credit Deduction Rules

When a booking is successfully confirmed:

Booking Confirmed

↓

Team Wallet Debit

↓

Ledger Entry

↓

Updated Team Balance

The Team balance must never become negative.

The wallet deduction and booking confirmation must be completed as one reliable backend transaction.

The blueprint explicitly defines this atomic requirement.

---

## 3.22 Participant Rules

Participants are recorded specifically for each booking.

Example:

Team Members = 10

Booking Participants = 6

The booking participant list does not modify the permanent Team membership list.

Before the session begins, permitted participants may be added or removed.

Once the session starts, the final participant list is locked.

---

## 3.23 Member Usage Attribution

After a Turf session is completed:

Actual booking credits consumed

÷

Final participant count

=

Member credit usage

Example:

Booking = 1,800 credits

Participants = 6

$1,800 \div 6$

=

300 credits usage per participant

This is an **insight/usage attribution**, not a transfer of credits.

The Team wallet remains the owner of all 1,800 credits.

---

## 3.24 Cancellation Rules

Cancellation eligibility is determined using the scheduled session time and authoritative server time.

### **Cancellation 24+ hours before session**

Booking Cancelled

↓

Full Team credits refunded

↓

Wallet restored

↓

Applicable discounted-hour usage restored

### **Cancellation within 24 hours**

Booking Cancelled

↓

No credit refund

The exact UI must communicate the policy clearly.

The backend, not the mobile device clock, determines eligibility.

---

## 3.25 Weekly Leaderboard Rules

The leaderboard period is:

**Monday 00:00 → Sunday 23:59**

using the configured Thrill Mill Club business timezone.

### Team Leaderboard

Rank by:

**Number of completed Turf bookings during the week**

### Member Leaderboard

Rank by:

**Credits used through completed participation during the week**

Leaderboards are calculated from completed historical records.

Self-entered scores and unverified match results do not affect ranking.

---

## 3.26 Team Chat Rules

Each Team has its own dedicated chat.

Only users with valid Team membership may access that Team's chat.

Chat is intended for:

- Turf coordination
- Game communication
- Team announcements
- Player coordination

The MVP provides basic moderation only.



Advanced AI moderation, media-heavy chat and social-network functionality remain outside the MVP.

---

## 3.27 Admin Operational Rules

Admin actions include:

- Creating Teams
- Reviewing membership requests
- Verifying external payment
- Activating memberships
- Loading Team credits
- Performing authorized credit adjustments
- Creating bookings on behalf of Teams
- Viewing all Teams
- Viewing booking activity
- Blocking/unblocking Turf slots
- Viewing operational analytics

Any Admin action affecting credits or booking history must create an audit record.

The Admin's platform-level authority must be enforced by backend authorization, not merely by hiding UI elements.

---

## 3.28 Core Data Integrity Rules

The MVP must always preserve these invariants:

### **Credit Integrity**

**Team credits cannot become negative.**

### **Booking Integrity**

**One Turf slot cannot have two confirmed bookings simultaneously.**

### **Membership Integrity**

**Only active membership provides normal Team participation and membership benefits.**

### **Payment Integrity**

**One verified membership activation cannot allocate the same membership credits twice.**

### **Ledger Integrity**

**Every Team credit balance change must have a corresponding ledger entry.**

### **Participant Integrity**

**Only eligible Team members can be recorded as participants.**

### **Cancellation Integrity**

**Refunds are issued only when the cancellation satisfies the 24-hour policy.**

### **Usage Integrity**

**Member usage is based on actual credits consumed divided by final completed participants.**

These invariants are consistent with the blueprint's defined data-integrity rules.

---

## **3.29 Business Rule Priority**

When rules conflict or an implementation decision must be made, prioritize:

1. Financial / Credit Correctness



2. Booking Integrity



3. Authorization / Security



4. Membership Rules



5. Participant Integrity



6. Usage Attribution



7. Notifications



8. Analytics / Leaderboards



9. UI Convenience

The application should never sacrifice financial, booking or authorization correctness merely to simplify the UI.

---

## 3.30 Final Business Rule Principle

The complete business operation should follow:

USER REQUEST



AUTHENTICATION



AUTHORIZATION



BUSINESS RULE VALIDATION



DATABASE TRANSACTION



FINAL STATE



LEDGER / AUDIT WHERE REQUIRED



UI REFLECTS FINAL STATE

The mobile application should **request and display**.

The backend should **validate, calculate, authorize and commit**.

The database should **record what actually happened**. This separation is a central principle already defined in the blueprint.

## 4. CORE WORKFLOWS

This section defines the major operational workflows of the **Thrill Mill Club MVP**.

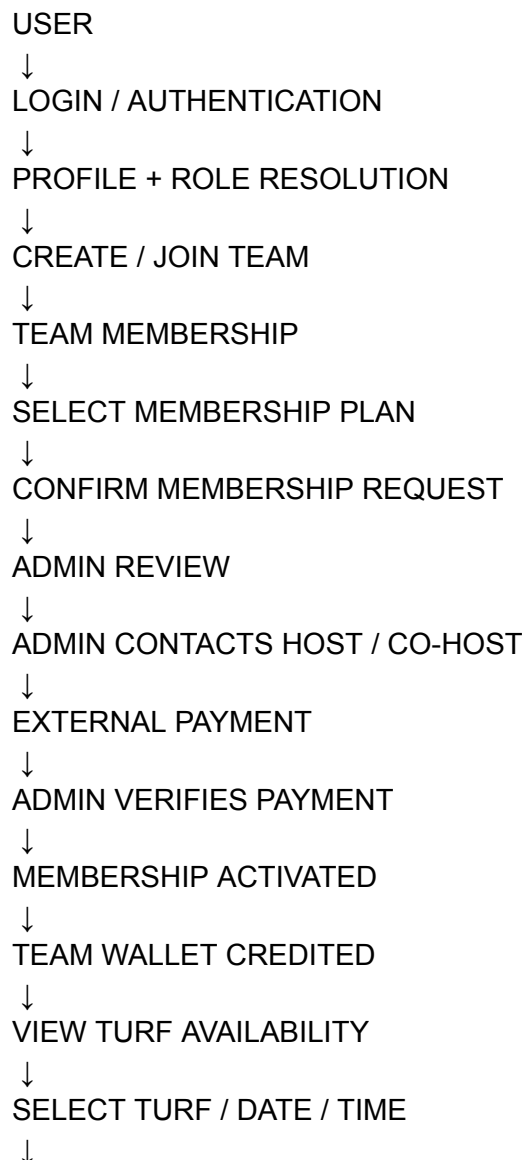
These workflows describe how users, Admins, the backend and the database interact from the beginning of an action until the final system state.

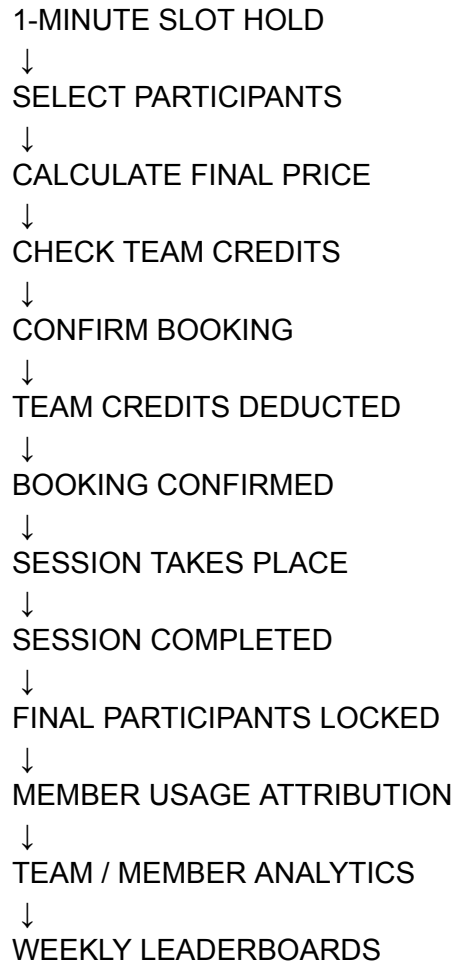
The workflows below are based on the existing blueprint, with the latest changes incorporated: **Team terminology, Admin-assisted Team creation, Admin membership approval with external payment collection, Admin-assisted booking, and removal of Razorpay from the membership-payment flow.**

---

## 4.1 Overall MVP Workflow

The complete Member/Team journey is:





The core booking and usage chain is retained from the original blueprint: Turf availability → hold → participants → credit validation → confirmation → completion → usage attribution → analytics/leaderboards.

---

## 4.2 Workflow 1 — User Authentication

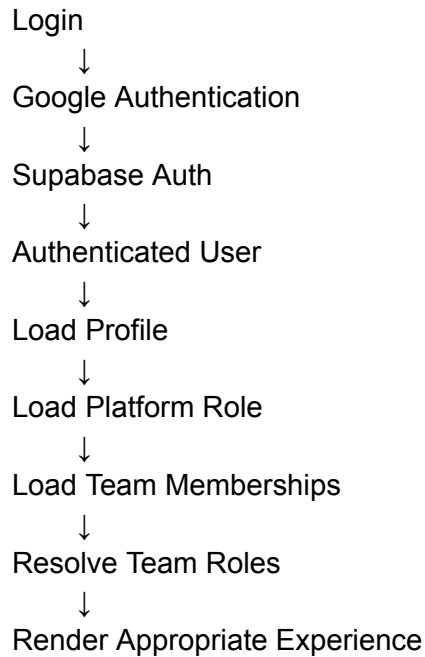
### Objective

Allow a user to securely enter the Thrill Mill Club application and establish their identity and permissions.

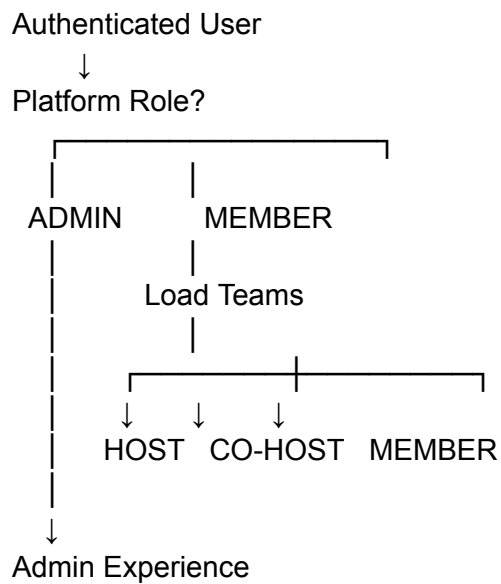
### Flow

Open Thrill Mill Club





### Role resolution



The same mobile application determines the appropriate experience after authentication rather than using separate applications.

---

## 4.3 Workflow 2 — Member Creates a Team

## Objective

Allow an eligible Member to create a reusable Team.

## Flow

Member  
↓  
Create Team  
↓  
Enter Team Name  
↓  
Add Members  
↓  
Assign Host / Co-host  
↓  
Select Membership Plan  
↓  
Review  
↓  
Confirm Team  
↓  
Team Created  
↓  
Team ID Generated  
↓  
Membership = PAYMENT\_PENDING

The creator may become the initial Host where the configured business rule permits.

The important updated point is that **Team creation and membership activation are now separate operations.**

---

## 4.4 Workflow 3 — Admin Creates a Team on Behalf of Customers

### Objective

Allow the Admin to create a Team for customers who register through the club, WhatsApp or direct communication.

## Flow

Admin



Create Team



Enter Team Details



Search Existing Members



Select Members



Assign Host



Assign Optional Co-host



Select Membership Plan



Review Team



Create Team



Team ID Generated



Membership = PAYMENT\_PENDING

The Admin-created Team is not a separate Team type.

After creation, it follows the exact same Team, wallet, membership and booking rules as a Member-created Team.

---

## 4.5 Workflow 4 — Add / Join Team Members

There are two supported paths.

### Member/Host-led invitation

Host / Co-host



Manage Team



↓  
Add Member  
↓  
Search Registered Member  
↓  
Select Member  
↓  
Send Invitation  
↓  
Member Notification  
↓  
Accept / Reject

Acceptance:

Accept  
↓  
Team Membership = ACTIVE  
↓  
Member Gets Team Access

Rejection:

Reject  
↓  
Membership Remains Inactive

A Member can belong to multiple Teams and membership is independent for each Team.

---

## 4.6 Workflow 5 — Admin Membership Request Review

### Objective

Allow the Admin to review and process a Team membership request.

This is the **new primary membership workflow**.

### Flow

Team

↓  
Members Added  
↓  
Membership Plan Selected  
↓  
Confirm Membership  
↓  
Membership Request Sent  
↓  
Admin Receives Request  
↓  
Admin Opens Request

The Admin sees:

Team Name  
Team ID  
Selected Membership Plan  
Team Members  
Host  
Host Phone Number  
Co-host  
Co-host Phone Number

Then:

Admin Reviews Team  
↓  
Contacts Host / Co-host  
↓  
Payment Collected Externally  
↓  
Admin Verifies Payment  
↓  
Approve Membership  
↓  
Activate Membership  
↓  
Load Team Credits  
↓  
Team = ACTIVE

The Host and Co-host contact details are specifically intended to make payment and membership coordination practical for the Admin.

---

## 4.7 Workflow 6 — External Membership Payment & Activation

### Objective

Process membership payment without an in-app payment gateway.

### Flow

Membership Request  
↓  
Payment Pending  
↓  
Admin Contacts Host / Co-host  
↓  
External Payment  
(UPI / QR / other approved method)  
↓  
Admin Confirms Payment Received  
↓  
Backend Validates Admin Permission  
↓  
Payment Record Created  
↓  
Membership Activated  
↓  
Team Wallet Credited  
↓  
Wallet Ledger Entry Created  
↓  
Audit Record Created

The mobile application must not directly modify the Team wallet.

The wallet update must occur through trusted backend logic.

---

## 4.8 Workflow 7 — Team Wallet

## Objective

Maintain the Team's credit balance and transaction history.

## Flow

Membership Activation



Team Wallet



Credits Available



Turf Booking



Credits Deducted



Wallet Ledger Updated

For refunds:

Eligible Cancellation



Booking Refund



Credits Returned



Ledger Updated



Team Balance Restored

The Team wallet remains the authoritative financial balance, while the ledger records the history of each credit event.

---

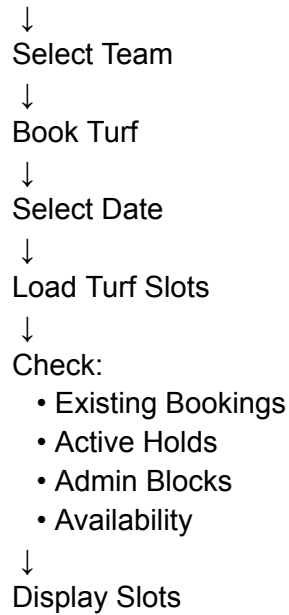
## 4.9 Workflow 8 — View Turf Availability

### Objective

Show bookable Turf inventory for the selected Team.

### Flow

User / Admin



Example:

6:00 PM → AVAILABLE  
7:00 PM → BOOKED  
8:00 PM → BLOCKED  
9:00 PM → AVAILABLE

Only valid available slots can be selected.

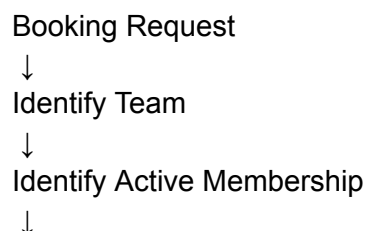
---

## 4.10 Workflow 9 — Calculate Booking Price

### Objective

Calculate the correct Turf price based on the Team's active membership and applicable usage.

### Flow



Determine Duration  
↓  
Check Rolling 24-Hour Usage  
↓  
Determine Remaining Discounted Hours  
↓  
Apply Membership / Standard Pricing  
↓  
Return Final Price

### **₹10,000 plan**

Maximum 3 discounted playing hours  
within rolling 24 hours

Example:

1st hour → ₹800  
2nd hour → ₹800  
3rd hour → ₹800  
4th hour → ₹1,000

### **₹25,000 plan**

Membership price → ₹800/hour  
No 3-hour restriction

These rules are already established in the blueprint.

---

## **4.11 Workflow 10 — Member / Host Booking**

### **Objective**

Allow an authorized Team user to initiate a Turf booking.

Host / Co-host  
↓  
Select Team  
↓  
Select Turf

↓  
Select Date  
↓  
Select Duration  
↓  
Select Time  
↓  
Calculate Price  
↓  
Select Participants  
↓  
Check Team Credits  
↓  
Create Booking Attempt

The member-facing booking flow retains the original participant-selection requirement.

---

## 4.12 Workflow 11 — Admin-Assisted Booking

### Objective

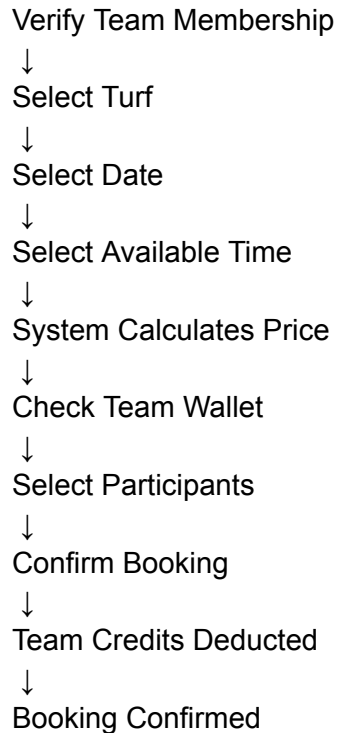
Allow the Admin to create a booking on behalf of a Team.

The Admin may receive a request through WhatsApp or another communication channel containing:

Team ID  
Preferred Date  
Preferred Time

### Flow

Admin  
↓  
Create Booking  
↓  
Enter / Search Team ID  
↓  
Load Team  
↓



The Admin must use the **same booking rules and backend booking engine** used by Host/Co-host bookings.

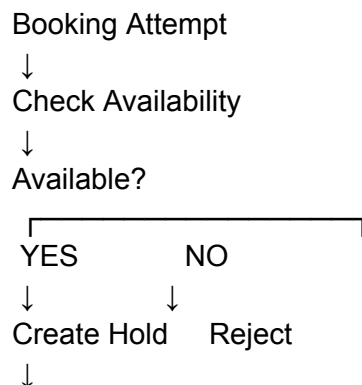
The Admin does not manually calculate the price or directly reduce the Team wallet.

---

## 4.13 Workflow 12 — One-Minute Slot Hold

### Objective

Prevent two users or Teams from confirming the same Turf slot simultaneously.





Slot = HELD



1-Minute Timer



Confirm Booking

If confirmation succeeds:

HELD



CONFIRMED

If the hold expires:

HELD



EXPIRED



AVAILABLE

The hold expiry must be determined by a server-controlled timestamp.

---

## 4.14 Workflow 13 — Participant Selection

The Host, Co-host or authorized Admin selects the Members participating in the particular Turf session.

Team



Member List



Select Participants



Review



Confirm

The participant list belongs to that specific booking.

Example:

Team Members = 10

Booking Participants = 6

The booking participant list does not change the permanent Team membership list.

Before the session begins, permitted participant changes can be made.

Once the session starts, the participant list is locked.

---

## 4.15 Workflow 14 — Confirm Booking

After the hold and booking requirements are satisfied:

Booking Hold

↓

Validate:

- Hold Still Valid
- Slot Available
- Team Has Credits
- User Authorized
- Price Correct
- Participants Valid

↓

Deduct Team Credits

↓

Create Wallet Ledger Entry

↓

Create Booking

↓

Generate Booking ID

↓

Slot = CONFIRMED

↓

Send Notifications

The booking and wallet deduction must be committed as one reliable backend operation.

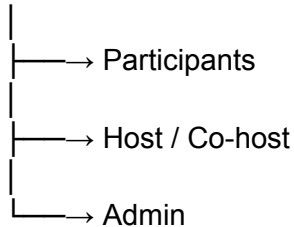
The original blueprint explicitly requires booking confirmation and credit deduction to be atomic.

---

## 4.16 Workflow 15 — Booking Notification

After successful booking:

Booking Confirmed



Notifications may communicate:

"Your Turf booking has been confirmed."

or:

"Weekend Warriors booked Turf for 8 PM."

Notifications are communication mechanisms only; the database remains the source of truth.

---

## 4.17 Workflow 16 — Modify Participants

Before the scheduled session:

Booking



Host / Co-host / Authorized Admin



Modify Participants



Validate Member



Update Booking Participants

Once the session starts:

SESSION START



Participant List LOCKED

The final participant list becomes historical data.

---

## 4.18 Workflow 17 — Session Completion

At the end of the scheduled Turf session:

Booking End Time



Booking Eligible for Completion



Final Participants Confirmed



Booking = COMPLETED



Usage Attribution



Member Statistics Updated



Team Analytics Updated



Leaderboard Data Updated

The final participant list is the basis for usage attribution.

---

## 4.19 Workflow 18 — Member Usage Attribution

This is an analytics calculation, not a credit transfer.

Completed Booking



Actual Credits Consumed



Final Participant Count



Credits Used Through Participation

Formula:

$$\frac{\text{Actual Booking Credits}}{\text{Final Participants}} = \text{Per-Member Usage}$$

Example:

$$\frac{1,800 \text{ credits}}{6 \text{ participants}} = 300 \text{ credits/member}$$

The Team wallet remains unchanged by this attribution calculation.

---

## 4.20 Workflow 19 — Mixed-Rate Booking Usage

A booking may contain both membership-priced and standard-priced hours.

Example:

Hour 1 → ₹800  
Hour 2 → ₹1,000  
-----  
Total → ₹1,800

After completion:

$$\frac{₹1,800}{\text{Final Participants}} = \text{Member Usage}$$

The attribution uses the **total actual Team credits consumed**, not separate attribution by rate.

---

## 4.21 Workflow 20 — Cancellation

User / Authorized Operator



Cancel Booking



Check Scheduled Start



Hours Remaining?

### **24 hours or more**

Eligible



Booking Cancelled



Full Credits Refunded



Team Wallet Restored



Applicable Discounted Hours Restored



History Updated



Notification

### **Less than 24 hours**

Late Cancellation



No Credit Refund



History Updated

The backend must determine eligibility using authoritative time.

---

## 4.22 Workflow 21 — ₹10,000 Discounted-Hour Restoration

This applies to the ₹10,000 plan.

Example:

Available Allowance = 3 hours

Booking A = 2 discounted hours

Usage = 2 / 3

If Booking A is cancelled at least 24 hours before the session:

Refund = 1,600 credits

↓

Restore 2 discounted hours

↓

Allowance = 0 / 3 used

This prevents refunded bookings from consuming the discounted-hour allowance.

---

## 4.23 Workflow 22 — Admin Turf Slot Blocking

Admin

↓

Turf Management

↓

Select Date / Time

↓

Block Slot

↓

Enter Reason

↓

Save

↓

Slot = BLOCKED

Possible reasons:

- Maintenance
- Private event
- Club closure
- Operational requirement

Once blocked, the slot cannot be selected for normal booking.

---

## 4.24 Workflow 23 — Club-Initiated Booking Cancellation

If an Admin blocks a slot that already contains a confirmed booking:

Admin Blocks Turf



Find Affected Booking



Booking Affected?



Cancel / Reschedule



Apply Configured Refund Policy



Notify Affected Users

This workflow should be controlled by the configured Club policy.

---

## 4.25 Workflow 24 — Admin Manual Credit Adjustment

For authorized operational adjustments:

Admin



Select Team



Credit Adjustment



Enter Amount



Enter Reason





Confirm



Backend Authorization



Update Team Wallet



Create Ledger Entry



Create Audit Record

The Admin must never directly overwrite the wallet balance.

---

## 4.26 Workflow 25 — Weekly Team Leaderboard

At the end of each week:

Monday 00:00



Collect Completed Turf Activity



Monday → Sunday



Count Completed Team Bookings



Rank Teams



Display Weekly Team Leaderboard

Only completed Turf bookings count.

---

## 4.27 Workflow 26 — Weekly Member Leaderboard

Monday 00:00



Collect Completed Participation  
↓  
Calculate Member Usage  
↓  
Aggregate Credits Used Through Participation  
↓  
Rank Members  
↓  
Display Weekly Member Leaderboard

No manually submitted match scores are used.

---

## 4.28 Workflow 27 — Admin Membership Payment Failure / Delay

Because payment is externally collected, the system must support a request remaining pending.

Membership Request  
↓  
PAYMENT\_PENDING  
↓  
Admin Reviewing  
↓  
Payment Not Yet Received  
↓  
Remain PAYMENT\_PENDING

The Team should not become active and credits should not be loaded until the Admin completes the verification process.

---

## 4.29 Workflow 28 — Admin Membership Approval Success

Admin Reviews Request  
↓  
Contacts Host / Co-host



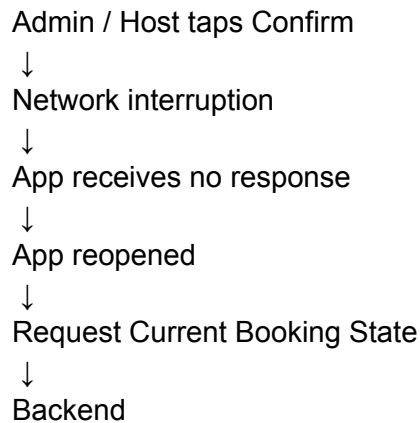
This becomes the primary replacement for the old Razorpay/webhook workflow.

---

## 4.30 Workflow 29 — Failed / Interrupted Booking

A mobile/network failure must not create inconsistent state.

Example:



Possible result:

CONFIRMED

or:

EXPIRED / FAILED

The application must not blindly create another booking because the first response was not received.

---

## 4.31 Workflow 30 — Repeat Booking

Where supported:

Previous Booking



Repeat Booking



Preload:

- Team
- Participants
- Duration
- Turf Preference



Select New Date / Time



Recheck Availability



Recalculate Current Price



New 1-Minute Hold



Confirm



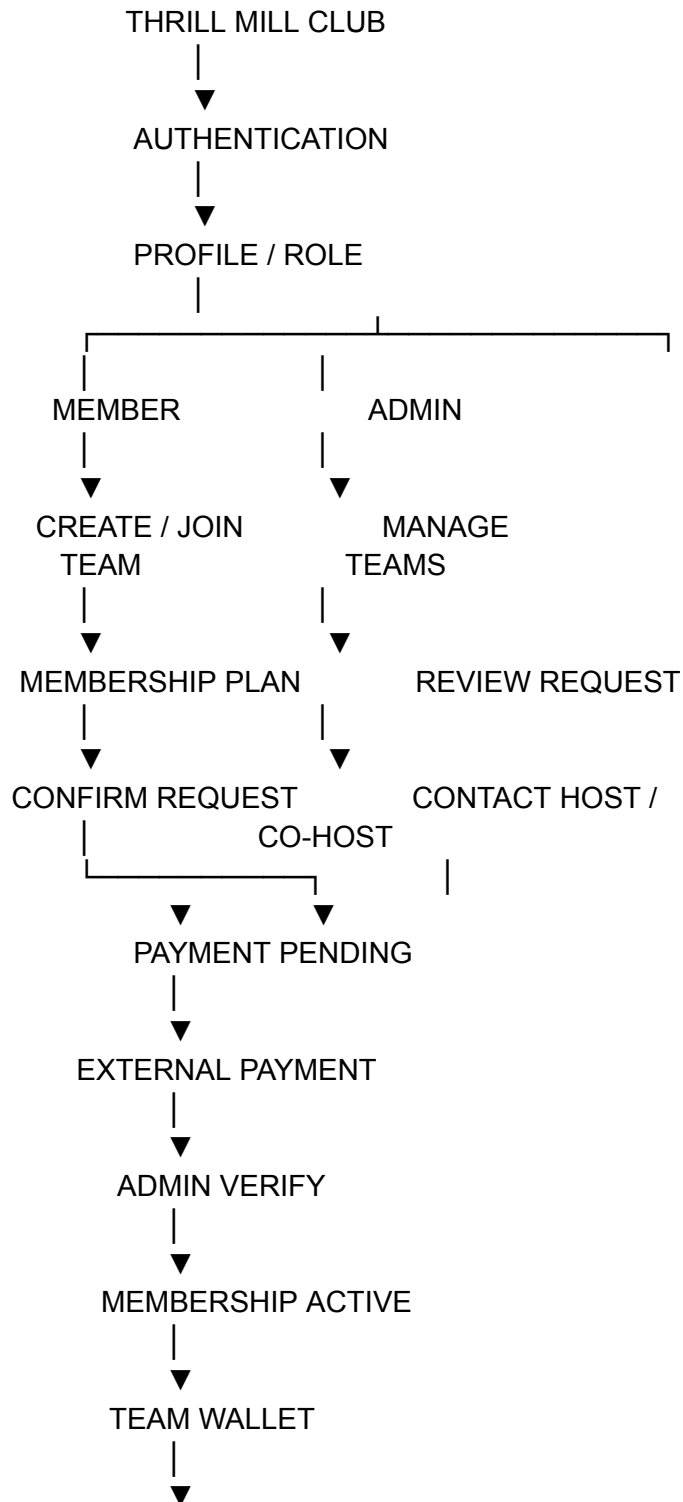
New Booking

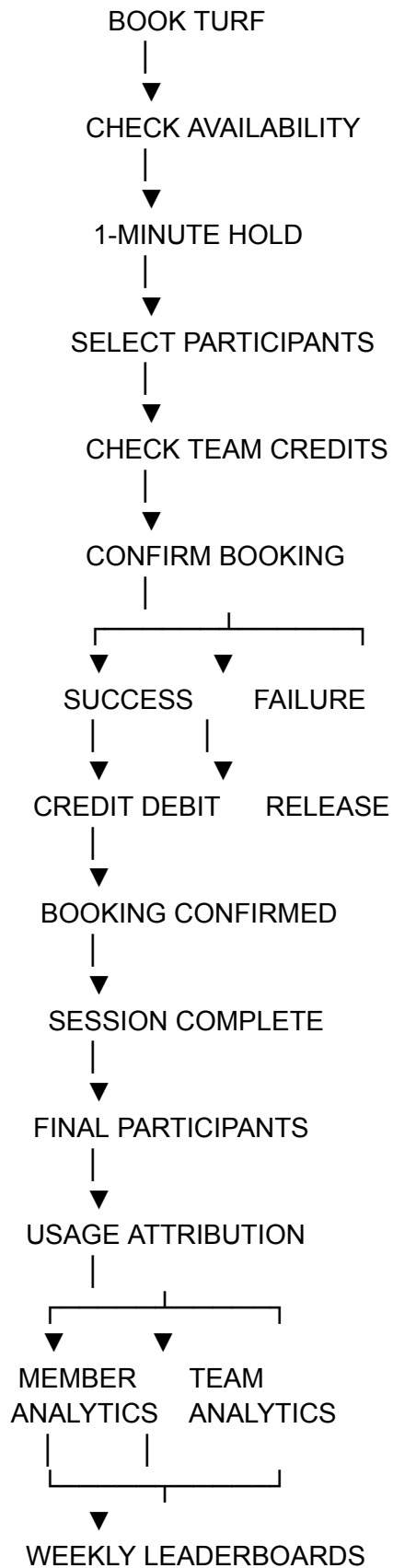
The previous booking's price must not be copied blindly; current membership and pricing rules must be recalculated.

---

## 4.32 Final MVP Operational Flow

The updated Thrill Mill Club MVP can therefore be summarized as:





The core invariant remains: the database records what actually happened, while analytics and leaderboards are derived from completed operational records.

## 5. STATE MACHINES

This section defines the valid states, transitions and conditions for the major entities in the **Thrill Mill Club MVP**.

A state machine ensures that an entity can only move through valid stages. Critical state transitions must be validated by the backend and, where they affect credits, bookings or Admin operations, committed atomically and recorded appropriately.

---

### 5.1 State-Machine Principles

The MVP follows these principles:

1. Every major entity has a clearly defined current state.
2. Only valid state transitions are permitted.
3. Transitions are triggered by defined Member, Host, Co-host, Admin or system actions.
4. Critical transitions are validated server-side.
5. A failed operation must not leave partially updated financial or booking state.
6. Financial and booking transitions must be atomic.
7. Historical records must not be silently rewritten.

The core principle is:

Action Requested



Current State Checked



Permission Checked



Business Rules Checked



Valid Transition



Database Updated



Audit / Ledger Where Required

This follows the blueprint's requirement that business-critical state transitions are enforced by the backend/database rather than being decided by the mobile UI.

---

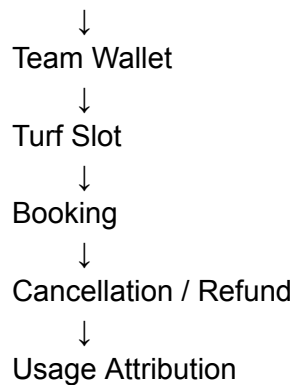
## 5.2 Entity State Overview

The major MVP state machines are:

1. Team
2. Team Membership
3. Membership Request / Activation
4. Payment Verification
5. Team Wallet / Credit
6. Turf Slot
7. Slot Hold
8. Booking
9. Booking Participant
10. Session
11. Cancellation / Refund
12. Chat Message
13. Notification
14. Leaderboard Activity

The highest-risk chain is:

Membership Payment Verification



---

## 5.3 Team State Machine

A Team may move through:



CREATED  
↓  
ACTIVE  
↓  
INACTIVE  
↓  
ARCHIVED

### State definitions

| State    | Meaning                                                        |
|----------|----------------------------------------------------------------|
| CREATED  | Team record has been created but setup may not yet be complete |
| ACTIVE   | Team is operational                                            |
| INACTIVE | Team is temporarily unavailable for normal operations          |
| ARCHIVED | Team is no longer active but historical records are retained   |

For the MVP, a newly created Team with a pending membership activation should remain a valid Team record while its membership remains pending.

---

## 5.4 Team Membership State Machine

Each user's relationship with a Team is independently tracked.

INVITED  
↓  
PENDING  
↓  
ACTIVE  
↓  
REMOVED

Alternative paths:

PENDING  
↓  
REJECTED

ACTIVE



LEFT

## State definitions

| State    | Meaning                                           |
|----------|---------------------------------------------------|
| INVITED  | Team invitation has been created                  |
| PENDING  | Membership awaits required user action/processing |
| ACTIVE   | User has valid Team access                        |
| REJECTED | Invitation was declined                           |
| REMOVED  | User's Team access was revoked                    |
| LEFT     | User voluntarily left the Team                    |

Only an **ACTIVE** Team membership should provide normal Team access. Historical booking and usage records remain available after a membership becomes REMOVED or LEFT.

---

## 5.5 Membership Request / Activation State Machine

This is the primary membership lifecycle for the updated MVP.

PLAN\_SELECTED



REQUEST\_SUBMITTED



PAYMENT\_PENDING



ADMIN\_REVIEW



PAYMENT\_VERIFIED



ACTIVE

Possible failure path:

ADMIN\_REVIEW



PAYMENT\_REJECTED / NOT\_VERIFIED



PAYMENT\_PENDING

### Important transition

PAYMENT\_VERIFIED



MEMBERSHIP\_ACTIVE



TEAM\_CREDITS\_ALLOCATED

The Admin must not be able to activate a membership without completing the required payment-verification process.

---

## 5.6 External Payment Verification State Machine

Because the MVP no longer uses Razorpay/in-app payment processing, payment state is based on the **external payment + Admin verification process**.

PAYMENT\_EXPECTED



PAYMENT\_PENDING



ADMIN\_VERIFICATION



VERIFIED

Alternative:

ADMIN\_VERIFICATION



NOT\_VERIFIED



PAYMENT\_PENDING

### Important rule

Only:

VERIFIED

can trigger:

Membership Activation



Credit Allocation

The payment record should identify:

- Team
  - Membership
  - Amount
  - Payment method
  - External reference where available
  - Admin/operator
  - Verification timestamp
  - Verification status
- 

## 5.7 Membership Activation → Credit Allocation

The critical financial transition is:

External Payment Received



Admin Verifies Payment



Backend Authorization Check



Payment = VERIFIED



Membership = ACTIVE



Create Wallet Credit Ledger



Increase Team Available Credits

These changes must be handled as one controlled backend transaction.

A repeated activation request must not generate duplicate credits.

Example:

Admin activates

↓

₹10,000 credits allocated

Admin accidentally activates again

↓

Backend detects membership already ACTIVE

↓

No second allocation

This replaces the old webhook-based duplicate protection in the original blueprint. The underlying requirement—preventing duplicate credit allocation—remains.

---

## 5.8 Team Wallet State Machine

Team credits follow:

CREDIT AVAILABLE

↓

CREDIT RESERVED

↓

CREDIT CONSUMED

Alternative:

CREDIT RESERVED

↓

CREDIT RELEASED

↓

CREDIT AVAILABLE

Refund path:

CREDIT CONSUMED

↓

ELIGIBLE BOOKING REFUND

↓

CREDIT REFUNDED

↓  
CREDIT AVAILABLE

The Team wallet remains the authoritative credit balance.

The ledger records each financial event rather than relying only on a mutable displayed balance.

---

## 5.9 Wallet Transaction State Machine

A wallet transaction may follow:

PENDING  
↓  
POSTED

Alternative:

PENDING  
↓  
FAILED

For booking-related reserved credits:

RESERVED  
↓  
CONSUMED

or:

RESERVED  
↓  
RELEASED

Once a financial transaction has been posted, it must not be arbitrarily rewritten into another transaction type.

---

## 5.10 ₹10,000 Membership Pricing Allowance State

The ₹10,000 plan has a separate usage state based on the rolling 24-hour discounted-hour allowance.

0 / 3

↓

1 / 3

↓

2 / 3

↓

3 / 3

↓

STANDARD PRICING

This represents **discounted playing hours used within the applicable rolling 24-hour window**.

It is not based on:

- Number of bookings
- Number of members
- Calendar day

Example:

2-hour booking

+

1-hour booking

=

3 / 3

The next playing hour is charged at the standard rate.

If an eligible booking is cancelled, the applicable discounted-hour usage is restored.

---

## 5.11 Turf Slot State Machine

A Turf slot can move through:

AVAILABLE  
↓  
HELD  
↓  
CONFIRMED

Alternative:

HELD  
↓  
EXPIRED  
↓  
AVAILABLE

Admin path:

AVAILABLE  
↓  
BLOCKED  
↓  
AVAILABLE

### State definitions

| State     | Meaning                                                  |
|-----------|----------------------------------------------------------|
| AVAILABLE | Slot can be selected                                     |
| HELD      | Temporarily reserved during an active booking attempt    |
| CONFIRMED | Slot has been successfully booked                        |
| EXPIRED   | Temporary hold expired                                   |
| BLOCKED   | Slot is unavailable because of an Admin/club restriction |

The original blueprint defines the same slot lifecycle and Admin block path.

---

## 5.12 One-Minute Slot Hold State Machine

AVAILABLE  
↓  
CREATE HOLD



↓  
HELD  
↓  
Start 1-Minute Timer

Two outcomes:

### **Successful confirmation**

HELD  
↓  
Validate Booking  
↓  
CONFIRMED

### **Timeout**

HELD  
↓  
1 Minute Expires  
↓  
EXPIRED  
↓  
AVAILABLE

The expiration timestamp must be determined by the backend/server.

The mobile timer is only a visual countdown and cannot extend or validate the hold.

---

## **5.13 Booking State Machine**

A booking follows:

HOLDING  
↓  
CONFIRMED  
↓  
IN\_PROGRESS  
↓  
COMPLETED

Cancellation path:

HOLDING



CANCELLED / EXPIRED

or:

CONFIRMED



CANCELLED

### **Possible terminal states**

COMPLETED

CANCELLED

FAILED

EXPIRED

An expired or cancelled booking must never return to CONFIRMED.

The booking state machine in the original blueprint follows the same core progression.

---

## **5.14 Booking Confirmation Transition**

The critical transition is:

HOLDING



Validate:

Hold valid

+

Slot available

+

Team credits sufficient

+

User authorized

+

Price valid

+

Participants valid



Deduct Team Credits



Create Wallet Ledger Entry



Booking = CONFIRMED



Slot = CONFIRMED

All critical changes should succeed together.

If any validation fails:

NO CONFIRMATION



NO CREDIT CONSUMPTION



HOLD RELEASED / EXPIRES

---

## 5.15 Booking Participant State Machine

Each participant relationship can follow:

SELECTED



LOCKED



COMPLETED



USAGE ATTRIBUTION

Before the session starts:

SELECTED



ADD / REMOVE ALLOWED

At session start:

FINAL PARTICIPANTS



LOCKED

After session completion, the final participant list becomes historical data.

---

## 5.16 Session State Machine

The Turf session follows:

SCHEDULED  
↓  
IN\_PROGRESS  
↓  
COMPLETED

Once completed:

COMPLETED  
↓  
Final Participants  
↓  
Usage Attribution  
↓  
Analytics

Historical completed session data should not be silently rewritten.

---

## 5.17 Cancellation / Refund State Machine

### Eligible cancellation

CONFIRMED  
↓  
Cancellation Request  
↓  
Check Server Time  
↓  
≥ 24 Hours Before Start?  
↓  
YES  
↓  
CANCELLED  
↓  
FULL CREDIT REFUND  
↓  
RESTORE DISCOUNTED-HOUR USAGE

## **Late cancellation**

CONFIRMED  
↓  
Cancellation Request  
↓  
< 24 Hours  
↓  
CANCELLED  
↓  
NO CREDIT REFUND

The backend must determine the 24-hour condition using authoritative time.

---

## **5.18 Admin Turf Block State Machine**

AVAILABLE  
↓  
ADMIN BLOCK  
↓  
BLOCKED

Unblock:

BLOCKED  
↓  
ADMIN UNBLOCK  
↓  
AVAILABLE

If a block affects an existing confirmed booking:

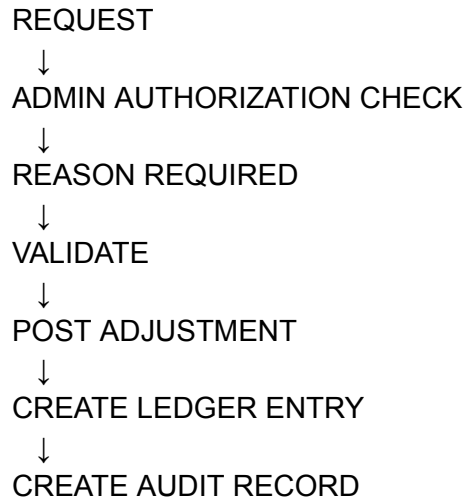
CONFIRMED BOOKING  
↓  
TURF BLOCKED  
↓  
AFFECTED BOOKING  
↓  
CANCEL / RESCHEDULE  
↓  
APPLY CONFIGURED POLICY

New bookings must not be allowed against a blocked slot.

---

## 5.19 Admin Credit Adjustment State Machine

For an authorized manual adjustment:



The Admin must not directly overwrite the Team balance.

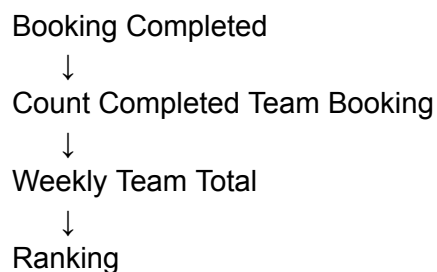
The original blueprint requires both a ledger event and an audit record for such adjustments.

---

## 5.20 Leaderboard Activity State

Leaderboard calculations are derived from completed records.

### Team



## Member

Participation Completed



Calculate Credits Used Through Participation



Weekly Member Total



Ranking

Unconfirmed bookings, cancelled bookings and arbitrary client-side values must not contribute to the leaderboard.

---

## 5.21 Critical Invalid Transitions

The backend must reject invalid transitions such as:

COMPLETED



CONFIRMED ❌

EXPIRED HOLD



CONFIRMED ❌

CANCELLED BOOKING



CONFIRMED ❌

INSUFFICIENT CREDITS



SUCCESSFUL DEBIT ❌

PAYMENT NOT VERIFIED



CREDIT ALLOCATION ❌

BLOCKED SLOT



NEW CONFIRMED BOOKING ❌

NON-ADMIN



ADMIN OPERATION ❌

These protections are explicitly identified in the original state-machine specification.

---

## 5.22 Highest-Priority State Machines for MVP

Because the MVP is intended to remain lean, the highest engineering attention should go to:

1. Membership Payment Verification



2. Team Wallet / Credit



3. Turf Slot



4. Booking



5. ₹10,000 Rolling 24-Hour Pricing



6. Cancellation / Refund



7. Participant Finalization



8. Usage Attribution

The original blueprint similarly prioritizes the wallet → booking → cancellation → usage chain because errors there can affect financial records and booking inventory.

---

## 5.23 State Transition Implementation Rule



Every state transition must answer four questions:

### **1. Who can trigger it?**

Member, Host, Co-host, Admin or system.

### **2. What conditions must be true?**

Current state, authorization, availability, balance, timing and relevant business rules.

### **3. What database changes happen?**

The exact records that must be updated.

### **4. What happens if the operation fails?**

No partial financial or booking state.

For example:

HELD → CONFIRMED

Who?

Host / Co-host / Admin

Conditions:

Hold valid

+

Slot available

+

Team credits sufficient

+

Price valid

+

User authorized

+

Participants valid

Database:

Booking = CONFIRMED

Slot = CONFIRMED

Wallet = deduct

Ledger = create debit

Failure:

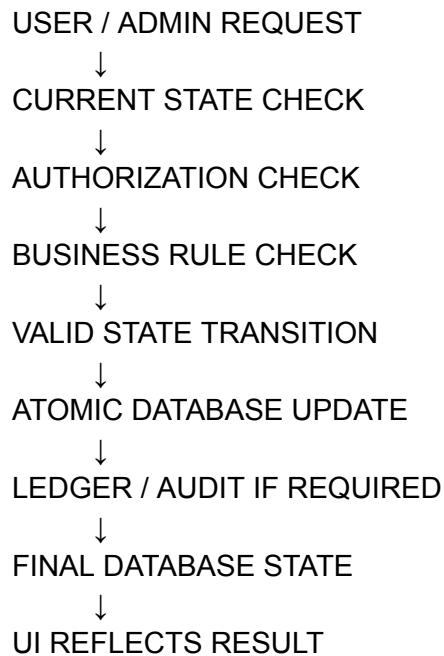
Booking remains unconfirmed  
Credits remain unchanged

This follows the state-transition approach specified in the source blueprint.

---

## 5.24 Final State-Machine Principle

The entire Thrill Mill Club MVP should follow:



The mobile application should never independently decide that:

- A payment is verified
- Credits were allocated
- A Turf slot is available
- A booking is confirmed
- A refund is valid
- A leaderboard score is correct

Those decisions belong to the backend and database. The original blueprint defines this same principle: the client requests, the backend validates and commits, and the database records the authoritative result.

# 6. DATA / DATABASE DESIGN

This section defines the database structure for the **Thrill Mill Club MVP**.

The database is the **single source of truth** for users, Teams, Team roles, memberships, credits, Turf inventory, bookings, participants, usage, chat, notifications, Admin actions and leaderboard data. The source blueprint recommends **PostgreSQL managed through Supabase**, with Supabase Auth, realtime capabilities and server-side functions around the database.

The original blueprint contains older “Network” terminology and Razorpay fields. For the final MVP, those should be interpreted as **Team** and **external/Admin-verified payment** respectively. No Razorpay-specific database dependency is required.

---

## 6.1 Database Design Goals

The database must prioritize:

### **Correctness**

No duplicate bookings, double credit deductions, duplicate activation allocations or invalid participant records.

### **Auditability**

Financial and sensitive Admin operations must be traceable.

### **Security**

A user's access must be determined by authenticated identity, platform role and Team membership.

### **Transparency**

A Team's credit balance and historical usage must be explainable from recorded transactions.

### **Historical integrity**

Completed bookings, participation, payments and ledger events should remain historically reliable.

### **Simplicity**

The MVP should use one primary database and avoid unnecessary services or duplicate stores. These goals are consistent with the source blueprint's database design objectives.

---

## 6.2 Database Technology

Recommended MVP stack:

React Native + Expo



Supabase



PostgreSQL

Supabase provides:

Authentication

Database

Row Level Security

Realtime capabilities

Server-side functions

The architecture should maintain:

ONE DATABASE



ONE SOURCE OF TRUTH



Teams

Members

Wallets

Bookings

Payments

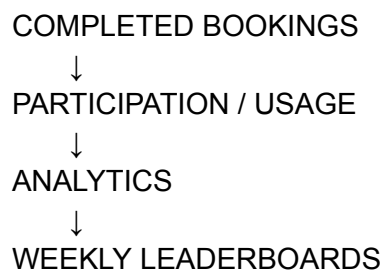
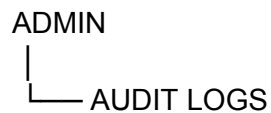
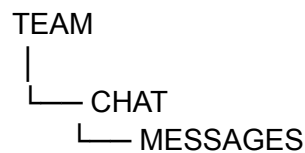
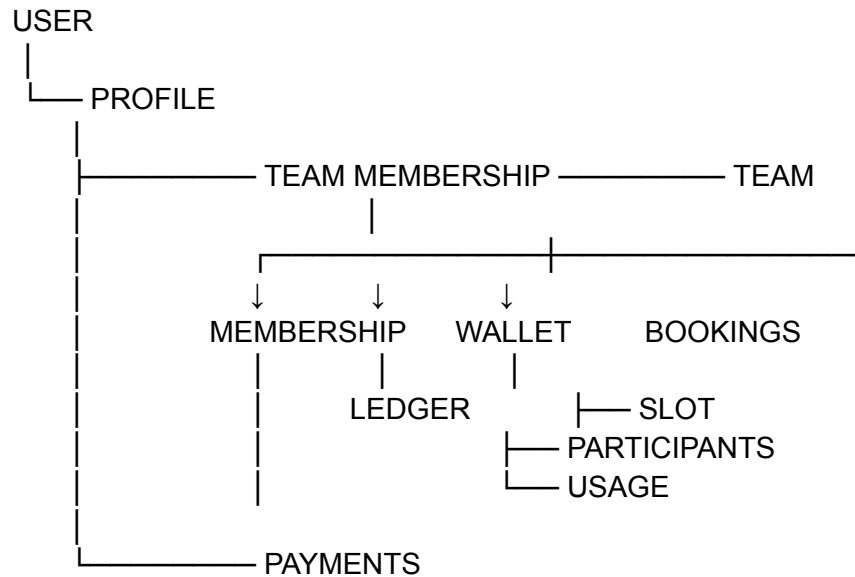
Analytics

The source blueprint explicitly recommends avoiding multiple independent copies of critical business data.

---

## 6.3 Core Data Architecture

The high-level relationship is:



The underlying source architecture follows the same core relationships between profiles, Team membership, Team wallet, bookings, participants, usage, chat, payments and leaderboard data.

---

## 6.4 Core Tables

The MVP should contain these primary tables:

| Table                    | Purpose                                        |
|--------------------------|------------------------------------------------|
| profiles                 | Application user profile                       |
| teams                    | Sports/community Team                          |
| team_members             | User ↔ Team relationship and Team role         |
| membership_plans         | Available membership plans                     |
| team_memberships         | Active/pending membership attached to Team     |
| team_wallets             | Current Team credit balance                    |
| wallet_ledger            | Immutable credit transaction history           |
| payments                 | External payment and Admin verification record |
| turf_resources           | Turf inventory                                 |
| turf_slots               | Bookable time slots                            |
| slot_holds               | Temporary one-minute booking holds             |
| bookings                 | Booking lifecycle and pricing snapshot         |
| booking_participants     | Members participating in a booking             |
| member_usage_attribution | Completed participation usage                  |
| chat_rooms               | Team chat room                                 |
| chat_messages            | Team chat messages                             |
| notifications            | In-app notification records                    |
| admin_audit_logs         | Sensitive Admin operation history              |
| leaderboard_weekly       | Derived weekly leaderboard results             |

The source blueprint identifies the same conceptual entity set, with the old naming adapted here to the final Team model.

---

## 6.5 profiles

Represents the authenticated application's user profile.

### Core fields

profiles

-----

id

auth\_user\_id

full\_name

mobile\_number

email

profile\_image\_url

date\_of\_birth / age\_band [optional]

platform\_role

created\_at

updated\_at

### Platform role

MEMBER

ADMIN

The Team-specific role should **not** be stored here.

For example:

User X

Platform Role = MEMBER

Team A → HOST

Team B → MEMBER

Team C → CO-HOST

This is important because Team roles are contextual to each Team. The source blueprint explicitly requires this separation.

---

## 6.6 teams

Represents a sports/community Team.

## Core fields

teams

-----

id

team\_code

name

created\_by

status

created\_at

updated\_at

## Status

ACTIVE

INACTIVE

ARCHIVED

## Relationships

TEAM

- |— TEAM MEMBERS
- |— TEAM MEMBERSHIP
- |— TEAM WALLET
- |— BOOKINGS
- |— CHAT
- |— ANALYTICS

**team\_code** / Team ID should be unique because Admin workflows may use it to search for a Team.

The source model defines a reusable group entity with members, membership, wallet, bookings, chat and analytics relationships.

---

## 6.7 team\_members

This is one of the most important tables because it represents the relationship between a user and a specific Team.

## Core fields

team\_members

-----



id  
team\_id  
user\_id  
team\_role  
membership\_status  
invited\_by  
joined\_at  
created\_at  
updated\_at

### **Team role**

HOST  
CO\_HOST  
MEMBER

### **Membership status**

INVITED  
PENDING  
ACTIVE  
REJECTED  
REMOVED  
LEFT

A user can have multiple records across different Teams:

User A + Team A  
User A + Team B  
User A + Team C

The database should enforce:

UNIQUE(team\_id, user\_id)

so a user cannot have duplicate membership rows for the same Team.

---

## **6.8 membership\_plans**

Stores the configurable membership plans.

### **Core fields**

membership\_plans

-----

id  
name  
price  
credit\_allocation  
membership\_booking\_rate  
standard\_booking\_rate  
discounted\_hour\_limit  
discount\_window\_hours  
status  
created\_at  
updated\_at

### Current plans

₹10,000 Plan

-----

price = 10000  
membership\_booking\_rate = 800  
standard\_booking\_rate = 1000  
discounted\_hour\_limit = 3  
discount\_window\_hours = 24

₹25,000 Plan

-----

price = 25000  
membership\_booking\_rate = 800  
standard\_booking\_rate = 1000  
discounted\_hour\_limit = NULL  
discount\_window\_hours = NULL

The exact `credit_allocation` should remain configurable rather than being hard-coded into the application. The source blueprint explicitly recommends storing plan configuration in the database.

---

## 6.9 team\_memberships

Associates a membership plan with a Team.

### Core fields

team\_memberships

-----  
id  
team\_id  
plan\_id  
payment\_id  
status  
activated\_at  
created\_at  
updated\_at

### **Status**

PENDING  
ACTIVE  
INACTIVE

Important distinction:

TEAM MEMBERSHIP  
+  
TEAM WALLET

are related but are not the same thing.

An active membership determines the Team's pricing rules; the wallet represents the Team's actual available credits.

Credits do not expire under the current MVP rules, so the database should not automatically remove unused credits because of plan age.

---

## **6.10 team\_wallets**

Stores the Team's current credit state.

### **Core fields**

team\_wallets  
-----  
id  
team\_id  
available\_credits  
reserved\_credits  
created\_at

updated\_at

The source blueprint defines the same wallet structure and identifies the Team wallet as the current financial source of truth.

### **Critical rule**

The mobile application must never directly update:

available\_credits

reserved\_credits

Those values can only be changed through authorized backend operations.

---

## **6.11 wallet\_ledger**

This is the primary financial history table.

Every Team credit movement must create a ledger record.

### **Core fields**

wallet\_ledger

-----

id

team\_id

transaction\_type

amount

balance\_before

balance\_after

reference\_type

reference\_id

created\_by

created\_at

metadata

### **Transaction types**

CREDIT\_ALLOCATION

BOOKING\_DEBIT

BOOKING\_REFUND

ADMIN\_ADJUSTMENT

SYSTEM\_RELEASE

Example:

Transaction: LED-10284

Team: Team A

Type: BOOKING\_DEBIT

Balance Before: 10,000

Amount: -800

Balance After: 9,200

Reference:

Booking #TM-TF-1024

The key invariant is:

Every credit balance change

↓

Must have

↓

One corresponding ledger entry

The source blueprint explicitly requires this relationship and prohibits normal application workflows from directly overwriting wallet state without a ledger event.

---

## 6.12 payments

The final MVP uses **external payment + Admin verification**.

Therefore this table should **not** depend on Razorpay-specific fields.

### Core fields

payments

-----

id

team\_id

membership\_id

amount

currency

payment\_method

external\_reference  
status  
verified\_by  
verified\_at  
created\_at  
updated\_at  
metadata

### **Payment methods**

UPI  
QR  
BANK\_TRANSFER  
CASH  
OTHER

### **Payment status**

PENDING  
VERIFIED  
REJECTED

A payment record represents the payment evidence and its verification state.

It does not itself mean that credits have been allocated.

The correct relationship is:

PAYMENT = VERIFIED  
↓  
MEMBERSHIP = ACTIVE  
↓  
TEAM WALLET CREDIT ALLOCATION  
↓  
LEDGER ENTRY

The old source blueprint contains gateway fields and gateway events; those are no longer needed under the finalized external-payment model. The underlying requirement retained from the blueprint is that payment verification, credit allocation and ledger creation must be linked and protected against duplicate allocation.

---

## **6.13 turf\_resources**

Represents physical Turf inventory.

For the initial MVP, this could contain one resource:

Thrill Mill Turf

but the schema should allow multiple Turf resources later.

### Core fields

turf\_resources

-----

id

name

resource\_type

status

created\_at

updated\_at

### Status

ACTIVE

INACTIVE

The source blueprint follows the same approach: keep the initial Turf inventory simple while allowing future expansion.

---

## 6.14 turf\_slots

Represents bookable Turf time inventory.

### Core fields

turf\_slots

-----

id

resource\_id

date

start\_time

end\_time

duration\_minutes

base\_price

status

created\_at  
updated\_at

## Inventory status

AVAILABLE  
BLOCKED

Dynamic booking status should preferably be determined from:

slot  
+  
active hold  
+  
confirmed booking  
+  
Admin block

rather than unnecessarily modifying the underlying Turf inventory row.

This matches the source blueprint's recommendation.

---

## 6.15 slot\_holds

Represents the temporary one-minute reservation.

### Core fields

slot\_holds  
-----  
id  
slot\_id  
team\_id  
booking\_id  
held\_by  
held\_at  
expires\_at  
status

### Status

ACTIVE  
EXPIRED



RELEASED  
CONVERTED

### Hold rule

held\_at = 18:30:00  
expires\_at = 18:31:00

The backend determines whether the hold remains valid.

The client countdown is only visual.

---

## 6.16 bookings

Represents a specific Turf booking attempt and its final booking record.

### Core fields

bookings

-----

id

booking\_reference

team\_id

slot\_id

created\_by

status

membership\_plan\_id

total\_duration\_minutes

total\_credits

membership\_rate\_applied

standard\_rate\_applied

discounted\_hours\_used

standard\_hours\_used

confirmed\_at

started\_at

completed\_at

cancelled\_at

cancellation\_reason

created\_at  
updated\_at

### Booking status

CREATED  
HOLDING  
CONFIRMED  
IN\_PROGRESS  
COMPLETED  
CANCELLED  
EXPIRED

The source blueprint defines the same booking lifecycle and requires `total_credits` to represent the actual amount committed/consumed by the booking.

---

## 6.17 Booking Pricing Snapshot

This is a critical design requirement.

The system must store **what pricing was actually applied**, not only the current membership plan.

Example:

Booking Duration = 2 hours

Hour 1 = ₹800

Hour 2 = ₹1,000

Total = 1,800 credits

Even if the Admin later changes the future membership price, this historical booking should remain:

Total = 1,800

Therefore the booking should store:

membership\_rate\_applied  
standard\_rate\_applied  
discounted\_hours\_used

standard\_hours\_used  
total\_credits

This makes historical billing and analytics explainable.

---

## 6.18 booking\_participants

Stores the actual Team members participating in each booking.

### Core fields

booking\_participants

-----

id  
booking\_id  
user\_id  
participant\_status  
selected\_at  
locked\_at  
created\_at  
updated\_at

### Status

SELECTED  
CONFIRMED  
LOCKED  
REMOVED  
COMPLETED

The Team roster and the booking participant list must remain separate.

Example:

Team Members = 10  
Booking Participants = 6

The source blueprint explicitly separates `team_members` from `booking_participants` for this reason.

---

## 6.19 Final Participant Lock

When the Turf session starts:

BOOKING  
↓  
SESSION START  
↓  
FINAL PARTICIPANTS  
↓  
LOCKED

After that point, the participant list becomes historical data.

Later changes to the Team roster must not modify the historical participant list for that completed booking.

---

## 6.20 `member_usage_attribution`

Stores individual usage **for analytics only**.

### Core fields

`member_usage_attribution`

-----

`id`  
`booking_id`  
`team_id`  
`user_id`  
`total_booking_credits`  
`participant_count`  
`attributed_credits`  
`created_at`

Example:

Booking = 1,800 credits  
Participants = 6

$1,800 / 6 = 300$

Records:

Member A → 300

Member B → 300

Member C → 300

...

This is not a wallet transfer.

There is:

NO member → member credit movement

The Team wallet remains the actual credit owner.

---

## 6.21 Mixed-Rate Usage Storage

A booking may contain both membership-priced and standard-priced hours.

Example:

1 hour × ₹800

1 hour × ₹1,000

-----

Total = ₹1,800

With six participants:

1,800 / 6

= 300 credits/member

The database therefore records:

total\_booking\_credits = 1800

participant\_count = 6

attributed\_credits = 300

The usage calculation is based on the **actual total booking charge**, not separate rate calculations for each participant.

---

## 6.22 chat\_rooms

Each Team has one primary chat room in the MVP.

### Core fields

chat\_rooms

-----

id

team\_id

status

created\_at

updated\_at

### Status

ACTIVE

ARCHIVED

This intentionally keeps chat simple for the MVP.

---

## 6.23 chat\_messages

Stores Team chat messages.

### Core fields

chat\_messages

-----

id

chat\_room\_id

team\_id

sender\_id

message

message\_status

moderation\_status

created\_at

updated\_at

### Message status

SENT

BLOCKED  
REMOVED

### **Moderation status**

PENDING  
ALLOWED  
BLOCKED  
FLAGGED

The MVP can use lightweight moderation; advanced AI moderation is outside scope.

---

## **6.24 notifications**

Stores in-app notification records.

### **Core fields**

notifications

-----

id  
user\_id  
team\_id  
notification\_type  
title  
body  
reference\_type  
reference\_id  
read\_at  
created\_at

### **Example types**

TEAM\_INVITE  
BOOKING\_CREATED  
BOOKING\_CONFIRMED  
BOOKING\_CANCELLED  
MEMBERSHIP\_UPDATE  
ADMIN\_BOOKING\_ALERT  
CHAT\_ACTIVITY

Push delivery should remain separate from the notification database record.

The database tells us that an event occurred; push delivery only communicates that event.

---

## 6.25 admin\_audit\_logs

Records sensitive Admin operations.

### Core fields

admin\_audit\_logs

-----

id

admin\_user\_id

action\_type

target\_type

target\_id

reason

before\_data

after\_data

created\_at

### Example actions

TEAM\_CREATED

MEMBERSHIP\_VERIFIED

CREDITS\_ALLOCATED

ADMIN\_CREDIT\_ADJUSTMENT

SLOT\_BLOCKED

SLOT\_UNBLOCKED

BOOKING\_CANCELLED

ROLE\_CHANGED

MEMBER\_REMOVED

Example:

Action:

ADMIN\_CREDIT\_ADJUSTMENT

Team:

Team A

Amount:

+500



Reason:  
Promotional credit

Admin:  
Admin User

Timestamp:  
Recorded automatically

The source blueprint requires sensitive Admin operations to be recorded with the operator, target, reason and resulting state.

---

## 6.26 leaderboard\_weekly

The leaderboard table is **derived data**, not the underlying source of truth.

### Core fields

leaderboard\_weekly

-----

id  
leaderboard\_type  
period\_start  
period\_end  
rank  
entity\_type  
entity\_id  
metric\_value  
created\_at

### Types

TEAM  
MEMBER

### Team metric

completed\_turf\_bookings

### Member metric

credits\_used\_through\_completed\_participation

For example:

TEAM

Rank 1

Team A

12 completed bookings

and:

MEMBER

Rank 1

Member A

2,400 credits used

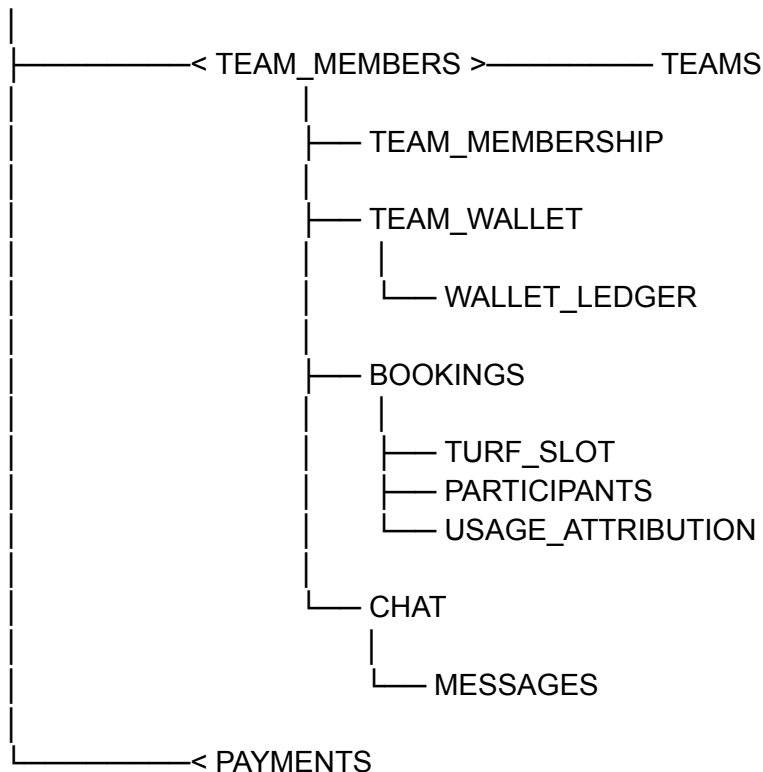
The underlying booking and usage records remain authoritative; this table is only a convenient weekly representation.

---

## 6.27 Main Relationships

The final ER relationship can be represented as:

PROFILES

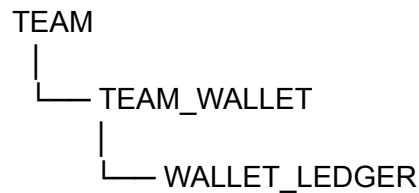


This follows the source blueprint's core relationship model while applying the final Team terminology.

---

## 6.28 Credit Ownership Relationship

The most important financial relationship is:



Not:



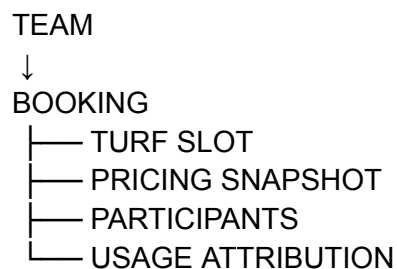
There are no individual Team-credit wallets in the MVP.

Member usage is an analytics record only.

---

## 6.29 Booking Relationship

Every booking should be traceable through:



This gives the system enough historical information to explain:

Why did this booking cost 1,800 credits?

The answer should be reconstructable from:

Active Team Plan



Eligible discounted hours



Booking duration



Discounted hours



Standard hours



Final booking total

The source blueprint specifically requires enough stored pricing data to reconstruct the booking cost.

---

## 6.30 Rolling 24-Hour Pricing Data

For the ₹10,000 membership, the database must make it possible to determine:

Current booking start



Previous qualifying completed/confirmed usage



Rolling 24-hour window



Discounted hours already consumed



Remaining discounted allowance



Current booking discounted hours



Standard hours



Final price

Example:

Allowance = 3 hours

Previous eligible usage = 2 hours

New booking = 2 hours

Remaining discounted = 1 hour

Therefore:

$1 \times ₹800$

$1 \times ₹1,000$

Total = ₹1,800

The final price must be calculated by the trusted backend and saved as the booking's historical pricing snapshot.

---

## 6.31 Recommended Indexes

Indexes should be based on actual application query patterns.

Initial important indexes:

`team_members(team_id, user_id)`

`team_members(user_id, membership_status)`

`bookings(team_id, status)`

`bookings(slot_id, status)`

`bookings(start_time)`

`bookings(created_at)`

`booking_participants(booking_id)`

`booking_participants(user_id)`

`wallet_ledger(team_id, created_at)`

`payments(team_id, status)`

chat\_messages(team\_id, created\_at)

notifications(user\_id, read\_at)

member\_usage\_attribution(user\_id, created\_at)

leaderboard\_weekly(period\_start, leaderboard\_type)

The source blueprint recommends these same query-oriented indexes and notes that the final index set should be validated against actual production query patterns.

---

## 6.32 Unique Constraints

The database should enforce business-critical uniqueness.

### **Team membership**

UNIQUE(team\_id, user\_id)

### **Booking reference**

UNIQUE(booking\_reference)

### **Team code**

UNIQUE(team\_code)

### **Payment verification / activation**

Instead of gateway IDs, the final implementation should enforce a suitable unique relationship preventing the same verified membership payment from generating multiple credit allocations.

For example, the activation transaction should be idempotent around:

membership\_id  
+  
verified payment  
+  
credit allocation

The source blueprint emphasizes uniqueness and idempotency for memberships, bookings and payment-driven credit allocation.

---

## 6.33 Foreign-Key Integrity

Database relationships should use foreign keys.

Examples:

```
bookings.team_id
    ↓
teams.id
booking_participants.booking_id
    ↓
bookings.id
booking_participants.user_id
    ↓
profiles.id
wallet_ledger.team_id
    ↓
teams.id
```

This prevents orphaned financial, booking and participant records. The source blueprint explicitly requires foreign-key integrity for this purpose.

---

## 6.34 Soft Delete vs Hard Delete

Critical historical records must not be hard-deleted.

Do not hard-delete:

```
wallet_ledger
payments
completed bookings
completed participation
admin_audit_logs
```

Use states such as:

```
CANCELLED
REMOVED
```

ARCHIVED

Temporary records such as expired slot holds may eventually be cleaned up after an appropriate retention period.

This preserves the historical record required for financial and operational auditing.

---

## 6.35 Database Transaction Boundaries

The most important workflows must be atomic.

### Booking confirmation

Check hold  
+  
Check availability  
+  
Check credits  
+  
Calculate price  
+  
Deduct Team wallet  
+  
Create ledger entry  
+  
Confirm booking

Either all succeed or none succeed.

### Eligible cancellation

Validate cancellation  
+  
Cancel booking  
+  
Refund Team credits  
+  
Create ledger entry  
+  
Restore discounted-hour usage



## Membership activation

Verify external payment  
+  
Record payment as VERIFIED  
+  
Activate membership  
+  
Allocate Team credits  
+  
Create ledger entry  
+  
Create audit record

The source blueprint explicitly defines these transaction boundaries and the rule that required changes must either all succeed or none do.

---

## 6.36 Source of Truth

The final source-of-truth map is:

| Data                   | Source of Truth                                  |
|------------------------|--------------------------------------------------|
| User identity          | Supabase Auth                                    |
| User profile           | <code>profiles</code>                            |
| Team membership        | <code>team_members</code>                        |
| Team role              | <code>team_members</code>                        |
| Active membership plan | <code>team_memberships</code>                    |
| Current credits        | <code>team_wallets</code>                        |
| Credit history         | <code>wallet_ledger</code>                       |
| External payment state | <code>payments</code>                            |
| Slot availability      | Turf inventory + holds + bookings + Admin blocks |
| Booking state          | <code>bookings</code>                            |

|               |                                                                     |
|---------------|---------------------------------------------------------------------|
| Participants  | <code>booking_participants</code>                                   |
| Member usage  | <code>member_usage_attribution</code>                               |
| Chat          | <code>chat_messages</code>                                          |
| Notifications | <code>notifications</code>                                          |
| Admin actions | <code>admin_audit_logs</code>                                       |
| Leaderboard   | Derived from completed records /<br><code>leaderboard_weekly</code> |

This follows the source blueprint's source-of-truth architecture, with the payment row updated for the final external-verification model.

---

## 6.37 What Must NOT Become Independent Truth

Do not duplicate the same business value across unrelated tables.

Bad:

`team_wallets.available_credits = 10,000`

`teams.credit_balance = 10,000`

`profiles.team_credit_balance = 10,000`

These can eventually disagree.

Prefer:

`team_wallets.available_credits`

as the current balance source.

Likewise, do not store manually editable leaderboard totals as the only source of truth.

The source blueprint explicitly recommends operational records rather than duplicate manually maintained totals.

---

## 6.38 Analytics as Derived Data

Most analytics should be generated from operational records.

For member usage:

Completed Booking  
↓  
Final Participants  
↓  
Usage Attribution  
↓  
Member Statistics  
↓  
Leaderboard

For Team credits:

Credit Allocation  
-  
Booking Debit  
+  
Booking Refund  
+  
Admin Adjustment  
=  
Explainable Team Balance

This makes analytics auditable and avoids manual statistics maintenance.

---

## 6.39 Database Security Boundary

Database access should follow:

Authentication  
↓  
User Identity  
↓

Platform Role / Team Membership



Row Level Security



Server-Side Validation



Database Transaction

Being authenticated does **not** automatically mean being authorized to perform an operation.

For example:

Authenticated Member

+

Not Host/Co-host



Cannot initiate Team booking

and:

Authenticated User

+

Not Admin

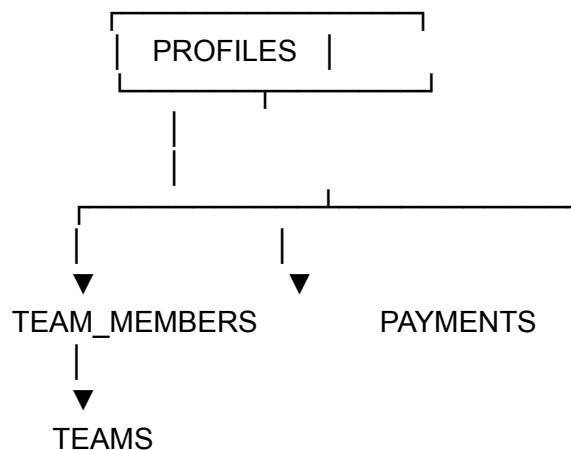


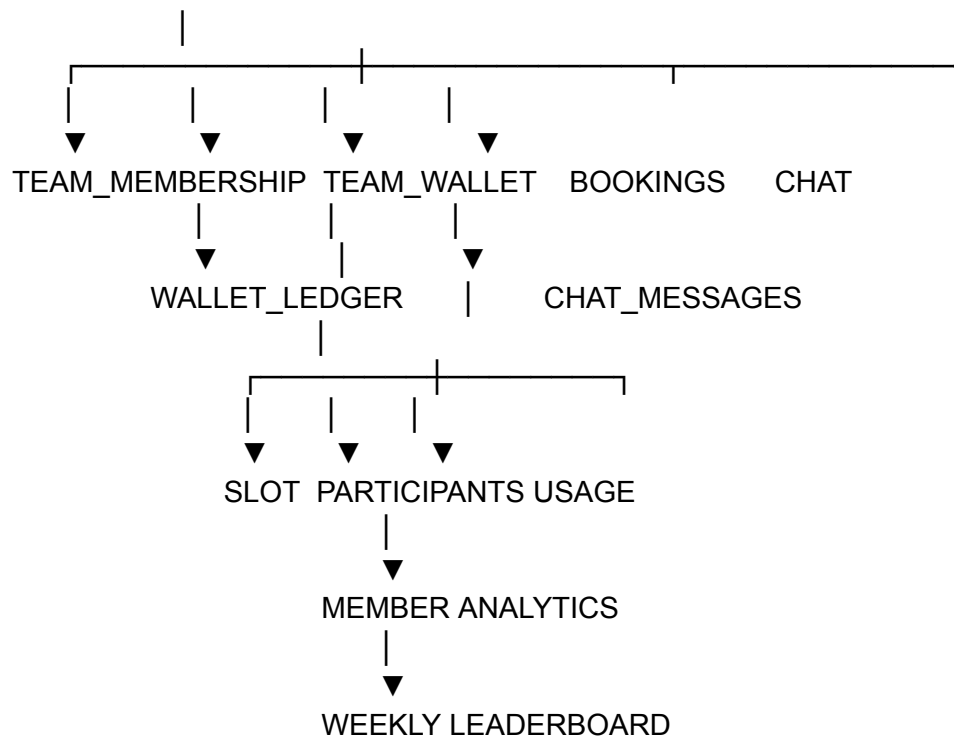
Cannot block Turf globally

The source blueprint defines this layered database security boundary explicitly.

---

## 6.40 Simplified Final ER Model





This is the simplified structure derived from the source blueprint's ER model.

---

## 6.41 Final Database Principle

The database should record **what actually happened**, not merely the latest totals displayed by the UI.

For example, instead of:

Member A  
credits\_used = 2,400

store:

Booking 1 → Member A → 300  
Booking 2 → Member A → 500  
Booking 3 → Member A → 400  
...

Then:

SUM(completed usage)

↓  
2,400

Likewise:

Team Credits

=

Allocations

- Booking Debits

+ Refunds

+ Admin Adjustments

This operational-event approach makes the system explainable, auditable and much safer for the credit/booking model.

**Final core chain:**

TEAM

↓

MEMBERSHIP

↓

WALLET

↓

LEDGER

↓

BOOKING

↓

PARTICIPANTS

↓

SESSION COMPLETION

↓

USAGE ATTRIBUTION

↓

ANALYTICS

↓

LEADERBOARDS

## 6. DATA / DATABASE DESIGN

This section defines the database structure for the **Thrill Mill Club MVP**.

The database is the **single source of truth** for users, Teams, Team roles, memberships, credits, Turf inventory, bookings, participants, usage, chat, notifications, Admin actions and leaderboard data. The source blueprint recommends **PostgreSQL managed through**

**Supabase**, with Supabase Auth, realtime capabilities and server-side functions around the database.

The original blueprint contains older “Network” terminology and Razorpay fields. For the final MVP, those should be interpreted as **Team** and **external/Admin-verified payment** respectively. No Razorpay-specific database dependency is required.

---

## 6.1 Database Design Goals

The database must prioritize:

### **Correctness**

No duplicate bookings, double credit deductions, duplicate activation allocations or invalid participant records.

### **Auditability**

Financial and sensitive Admin operations must be traceable.

### **Security**

A user's access must be determined by authenticated identity, platform role and Team membership.

### **Transparency**

A Team's credit balance and historical usage must be explainable from recorded transactions.

### **Historical integrity**

Completed bookings, participation, payments and ledger events should remain historically reliable.

### **Simplicity**

The MVP should use one primary database and avoid unnecessary services or duplicate stores. These goals are consistent with the source blueprint's database design objectives.

---

## 6.2 Database Technology

Recommended MVP stack:

React Native + Expo



Supabase



PostgreSQL

Supabase provides:

Authentication

Database

Row Level Security

Realtime capabilities

Server-side functions

The architecture should maintain:

ONE DATABASE



ONE SOURCE OF TRUTH



Teams

Members

Wallets

Bookings

Payments

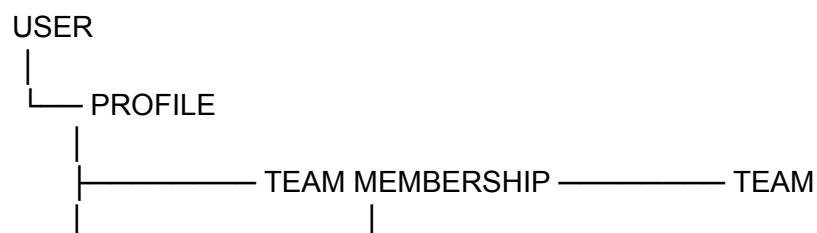
Analytics

The source blueprint explicitly recommends avoiding multiple independent copies of critical business data.

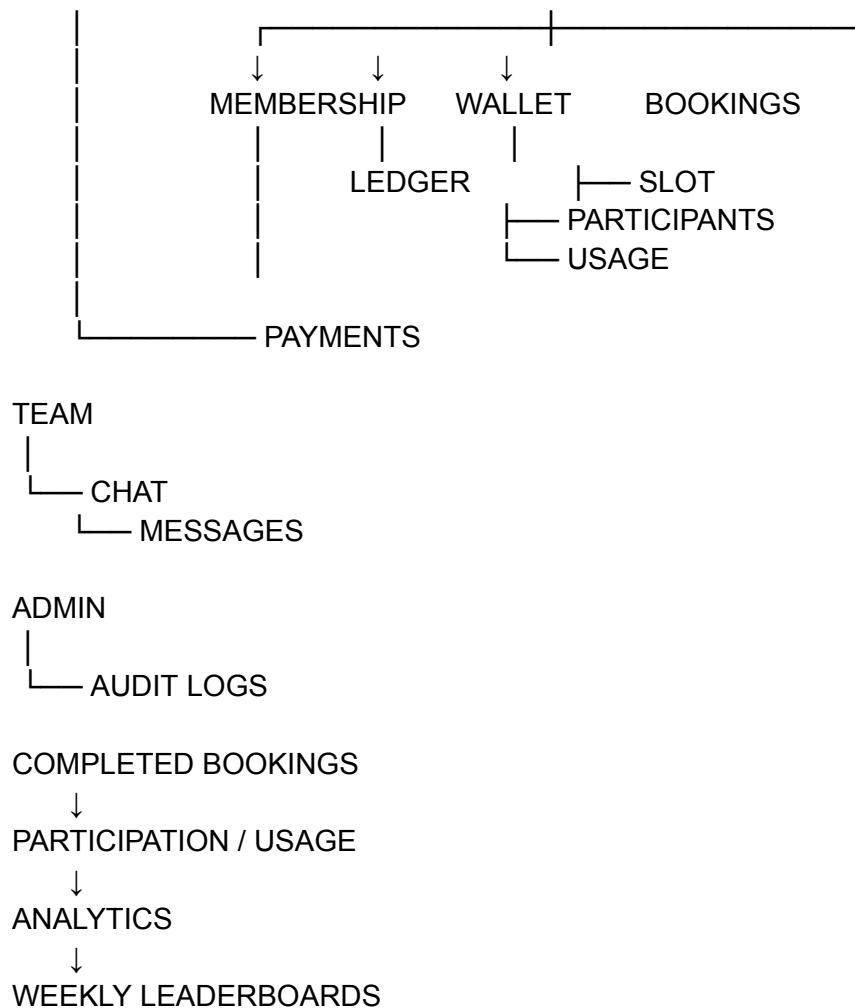
---

## 6.3 Core Data Architecture

The high-level relationship is:







The underlying source architecture follows the same core relationships between profiles, Team membership, Team wallet, bookings, participants, usage, chat, payments and leaderboard data.

---

## 6.4 Core Tables

The MVP should contain these primary tables:

| Table        | Purpose                                |
|--------------|----------------------------------------|
| profiles     | Application user profile               |
| teams        | Sports/community Team                  |
| team_members | User ↔ Team relationship and Team role |

|                                       |                                                |
|---------------------------------------|------------------------------------------------|
| <code>membership_plans</code>         | Available membership plans                     |
| <code>team_memberships</code>         | Active/pending membership attached to Team     |
| <code>team_wallets</code>             | Current Team credit balance                    |
| <code>wallet_ledger</code>            | Immutable credit transaction history           |
| <code>payments</code>                 | External payment and Admin verification record |
| <code>turf_resources</code>           | Turf inventory                                 |
| <code>turf_slots</code>               | Bookable time slots                            |
| <code>slot_holds</code>               | Temporary one-minute booking holds             |
| <code>bookings</code>                 | Booking lifecycle and pricing snapshot         |
| <code>booking_participants</code>     | Members participating in a booking             |
| <code>member_usage_attribution</code> | Completed participation usage                  |
| <code>chat_rooms</code>               | Team chat room                                 |
| <code>chat_messages</code>            | Team chat messages                             |
| <code>notifications</code>            | In-app notification records                    |
| <code>admin_audit_logs</code>         | Sensitive Admin operation history              |
| <code>leaderboard_weekly</code>       | Derived weekly leaderboard results             |

The source blueprint identifies the same conceptual entity set, with the old naming adapted here to the final Team model.

---

## 6.5 profiles

Represents the authenticated application's user profile.

## Core fields

profiles

-----

id

auth\_user\_id

full\_name

mobile\_number

email

profile\_image\_url

date\_of\_birth / age\_band [optional]

platform\_role

created\_at

updated\_at

## Platform role

MEMBER

ADMIN

The Team-specific role should **not** be stored here.

For example:

User X

Platform Role = MEMBER

Team A → HOST

Team B → MEMBER

Team C → CO-HOST

This is important because Team roles are contextual to each Team. The source blueprint explicitly requires this separation.

---

## 6.6 teams

Represents a sports/community Team.

## Core fields

teams

-----

id

team\_code  
name  
created\_by  
status  
created\_at  
updated\_at

### Status

ACTIVE  
INACTIVE  
ARCHIVED

### Relationships

TEAM  
|— TEAM MEMBERS  
|— TEAM MEMBERSHIP  
|— TEAM WALLET  
|— BOOKINGS  
|— CHAT  
|— ANALYTICS

**team\_code** / Team ID should be unique because Admin workflows may use it to search for a Team.

The source model defines a reusable group entity with members, membership, wallet, bookings, chat and analytics relationships.

---

## 6.7 team\_members

This is one of the most important tables because it represents the relationship between a user and a specific Team.

### Core fields

team\_members  
-----  
id  
team\_id  
user\_id  
team\_role

membership\_status  
invited\_by  
joined\_at  
created\_at  
updated\_at

### **Team role**

HOST  
CO\_HOST  
MEMBER

### **Membership status**

INVITED  
PENDING  
ACTIVE  
REJECTED  
REMOVED  
LEFT

A user can have multiple records across different Teams:

User A + Team A  
User A + Team B  
User A + Team C

The database should enforce:

UNIQUE(team\_id, user\_id)

so a user cannot have duplicate membership rows for the same Team.

---

## **6.8 membership\_plans**

Stores the configurable membership plans.

### **Core fields**

membership\_plans  
-----  
id  
name

price  
credit\_allocation  
membership\_booking\_rate  
standard\_booking\_rate  
discounted\_hour\_limit  
discount\_window\_hours  
status  
created\_at  
updated\_at

### Current plans

₹10,000 Plan

-----

price = 10000  
membership\_booking\_rate = 800  
standard\_booking\_rate = 1000  
discounted\_hour\_limit = 3  
discount\_window\_hours = 24

₹25,000 Plan

-----

price = 25000  
membership\_booking\_rate = 800  
standard\_booking\_rate = 1000  
discounted\_hour\_limit = NULL  
discount\_window\_hours = NULL

The exact `credit_allocation` should remain configurable rather than being hard-coded into the application. The source blueprint explicitly recommends storing plan configuration in the database.

---

## 6.9 team\_memberships

Associates a membership plan with a Team.

### Core fields

team\_memberships

-----

id  
team\_id  
plan\_id

payment\_id  
status  
activated\_at  
created\_at  
updated\_at

### **Status**

PENDING  
ACTIVE  
INACTIVE

Important distinction:

TEAM MEMBERSHIP  
+  
TEAM WALLET

are related but are not the same thing.

An active membership determines the Team's pricing rules; the wallet represents the Team's actual available credits.

Credits do not expire under the current MVP rules, so the database should not automatically remove unused credits because of plan age.

---

## **6.10 team\_wallets**

Stores the Team's current credit state.

### **Core fields**

team\_wallets  
-----  
id  
team\_id  
available\_credits  
reserved\_credits  
created\_at  
updated\_at

The source blueprint defines the same wallet structure and identifies the Team wallet as the current financial source of truth.

## Critical rule

The mobile application must never directly update:

available\_credits  
reserved\_credits

Those values can only be changed through authorized backend operations.

---

## 6.11 wallet\_ledger

This is the primary financial history table.

Every Team credit movement must create a ledger record.

### Core fields

wallet\_ledger

-----

id  
team\_id  
transaction\_type  
amount  
balance\_before  
balance\_after  
reference\_type  
reference\_id  
created\_by  
created\_at  
metadata

### Transaction types

CREDIT\_ALLOCATION  
BOOKING\_DEBIT  
BOOKING\_REFUND  
ADMIN\_ADJUSTMENT  
SYSTEM\_RELEASE

Example:

Transaction: LED-10284  
Team: Team A



Type: BOOKING\_DEBIT

Balance Before: 10,000

Amount: -800

Balance After: 9,200

Reference:

Booking #TM-TF-1024

The key invariant is:

Every credit balance change

↓

Must have

↓

One corresponding ledger entry

The source blueprint explicitly requires this relationship and prohibits normal application workflows from directly overwriting wallet state without a ledger event.

---

## 6.12 payments

The final MVP uses **external payment + Admin verification**.

Therefore this table should **not** depend on Razorpay-specific fields.

### Core fields

payments

-----

id

team\_id

membership\_id

amount

currency

payment\_method

external\_reference

status

verified\_by

verified\_at

created\_at

updated\_at

metadata

### **Payment methods**

UPI  
QR  
BANK\_TRANSFER  
CASH  
OTHER

### **Payment status**

PENDING  
VERIFIED  
REJECTED

A payment record represents the payment evidence and its verification state.

It does not itself mean that credits have been allocated.

The correct relationship is:

PAYMENT = VERIFIED  
↓  
MEMBERSHIP = ACTIVE  
↓  
TEAM WALLET CREDIT ALLOCATION  
↓  
LEDGER ENTRY

The old source blueprint contains gateway fields and gateway events; those are no longer needed under the finalized external-payment model. The underlying requirement retained from the blueprint is that payment verification, credit allocation and ledger creation must be linked and protected against duplicate allocation.

---

## **6.13 turf\_resources**

Represents physical Turf inventory.

For the initial MVP, this could contain one resource:

Thrill Mill Turf

but the schema should allow multiple Turf resources later.

### Core fields

turf\_resources

-----

id

name

resource\_type

status

created\_at

updated\_at

### Status

ACTIVE

INACTIVE

The source blueprint follows the same approach: keep the initial Turf inventory simple while allowing future expansion.

---

## 6.14 turf\_slots

Represents bookable Turf time inventory.

### Core fields

turf\_slots

-----

id

resource\_id

date

start\_time

end\_time

duration\_minutes

base\_price

status

created\_at

updated\_at

### Inventory status

AVAILABLE

BLOCKED

Dynamic booking status should preferably be determined from:

slot  
+  
active hold  
+  
confirmed booking  
+  
Admin block

rather than unnecessarily modifying the underlying Turf inventory row.

This matches the source blueprint's recommendation.

---

## 6.15 slot\_holds

Represents the temporary one-minute reservation.

### Core fields

slot\_holds  
-----  
id  
slot\_id  
team\_id  
booking\_id  
held\_by  
held\_at  
expires\_at  
status

### Status

ACTIVE  
EXPIRED  
RELEASED  
CONVERTED

### Hold rule

held\_at = 18:30:00

expires\_at = 18:31:00

The backend determines whether the hold remains valid.

The client countdown is only visual.

---

## 6.16 bookings

Represents a specific Turf booking attempt and its final booking record.

### Core fields

bookings

-----

id

booking\_reference

team\_id

slot\_id

created\_by

status

membership\_plan\_id

total\_duration\_minutes

total\_credits

membership\_rate\_applied

standard\_rate\_applied

discounted\_hours\_used

standard\_hours\_used

confirmed\_at

started\_at

completed\_at

cancelled\_at

cancellation\_reason

created\_at

updated\_at

### Booking status

CREATED  
HOLDING  
CONFIRMED  
IN\_PROGRESS  
COMPLETED  
CANCELLED  
EXPIRED

The source blueprint defines the same booking lifecycle and requires `total_credits` to represent the actual amount committed/consumed by the booking.

---

## 6.17 Booking Pricing Snapshot

This is a critical design requirement.

The system must store **what pricing was actually applied**, not only the current membership plan.

Example:

Booking Duration = 2 hours

Hour 1 = ₹800

Hour 2 = ₹1,000

Total = 1,800 credits

Even if the Admin later changes the future membership price, this historical booking should remain:

Total = 1,800

Therefore the booking should store:

membership\_rate\_applied  
standard\_rate\_applied  
discounted\_hours\_used  
standard\_hours\_used  
total\_credits

This makes historical billing and analytics explainable.

---

## 6.18 booking\_participants

Stores the actual Team members participating in each booking.

### Core fields

booking\_participants

-----

id  
booking\_id  
user\_id  
participant\_status  
selected\_at  
locked\_at  
created\_at  
updated\_at

### Status

SELECTED  
CONFIRMED  
LOCKED  
REMOVED  
COMPLETED

The Team roster and the booking participant list must remain separate.

Example:

Team Members = 10  
Booking Participants = 6

The source blueprint explicitly separates `team_members` from `booking_participants` for this reason.

---

## 6.19 Final Participant Lock

When the Turf session starts:

BOOKING  
↓  
SESSION START  
↓  
FINAL PARTICIPANTS  
↓  
LOCKED

After that point, the participant list becomes historical data.

Later changes to the Team roster must not modify the historical participant list for that completed booking.

---

## 6.20 member\_usage\_attribution

Stores individual usage **for analytics only**.

### Core fields

member\_usage\_attribution

-----

id  
booking\_id  
team\_id  
user\_id  
total\_booking\_credits  
participant\_count  
attributed\_credits  
created\_at

Example:

Booking = 1,800 credits  
Participants = 6

$1,800 / 6 = 300$

Records:

Member A → 300  
Member B → 300  
Member C → 300



...

This is not a wallet transfer.

There is:

NO member → member credit movement

The Team wallet remains the actual credit owner.

---

## 6.21 Mixed-Rate Usage Storage

A booking may contain both membership-priced and standard-priced hours.

Example:

1 hour × ₹800  
1 hour × ₹1,000  
-----  
Total = ₹1,800

With six participants:

1,800 / 6  
= 300 credits/member

The database therefore records:

total\_booking\_credits = 1800  
participant\_count = 6  
attributed\_credits = 300

The usage calculation is based on the **actual total booking charge**, not separate rate calculations for each participant.

---

## 6.22 chat\_rooms

Each Team has one primary chat room in the MVP.

### **Core fields**

chat\_rooms

-----

id

team\_id

status

created\_at

updated\_at

### **Status**

ACTIVE

ARCHIVED

This intentionally keeps chat simple for the MVP.

---

## **6.23 chat\_messages**

Stores Team chat messages.

### **Core fields**

chat\_messages

-----

id

chat\_room\_id

team\_id

sender\_id

message

message\_status

moderation\_status

created\_at

updated\_at

### **Message status**

SENT

BLOCKED

REMOVED

### **Moderation status**

PENDING

ALLOWED  
BLOCKED  
FLAGGED

The MVP can use lightweight moderation; advanced AI moderation is outside scope.

---

## 6.24 notifications

Stores in-app notification records.

### Core fields

notifications

-----

id  
user\_id  
team\_id  
notification\_type  
title  
body  
reference\_type  
reference\_id  
read\_at  
created\_at

### Example types

TEAM\_INVITE  
BOOKING\_CREATED  
BOOKING\_CONFIRMED  
BOOKING\_CANCELLED  
MEMBERSHIP\_UPDATE  
ADMIN\_BOOKING\_ALERT  
CHAT\_ACTIVITY

Push delivery should remain separate from the notification database record.

The database tells us that an event occurred; push delivery only communicates that event.

---

## 6.25 admin\_audit\_logs

Records sensitive Admin operations.

### Core fields

admin\_audit\_logs

-----

id  
admin\_user\_id  
action\_type  
target\_type  
target\_id  
reason  
before\_data  
after\_data  
created\_at

### Example actions

TEAM\_CREATED  
MEMBERSHIP\_VERIFIED  
CREDITS\_ALLOCATED  
ADMIN\_CREDIT\_ADJUSTMENT  
SLOT\_BLOCKED  
SLOT\_UNBLOCKED  
BOOKING\_CANCELLED  
ROLE\_CHANGED  
MEMBER\_REMOVED

Example:

Action:  
ADMIN\_CREDIT\_ADJUSTMENT

Team:  
Team A

Amount:  
+500

Reason:  
Promotional credit

Admin:  
Admin User

Timestamp:  
Recorded automatically

The source blueprint requires sensitive Admin operations to be recorded with the operator, target, reason and resulting state.

---

## 6.26 leaderboard\_weekly

The leaderboard table is **derived data**, not the underlying source of truth.

### Core fields

leaderboard\_weekly

-----

id  
leaderboard\_type  
period\_start  
period\_end  
rank  
entity\_type  
entity\_id  
metric\_value  
created\_at

### Types

TEAM  
MEMBER

### Team metric

completed\_turf\_bookings

### Member metric

credits\_used\_through\_completed\_participation

For example:

TEAM

Rank 1  
Team A  
12 completed bookings

and:

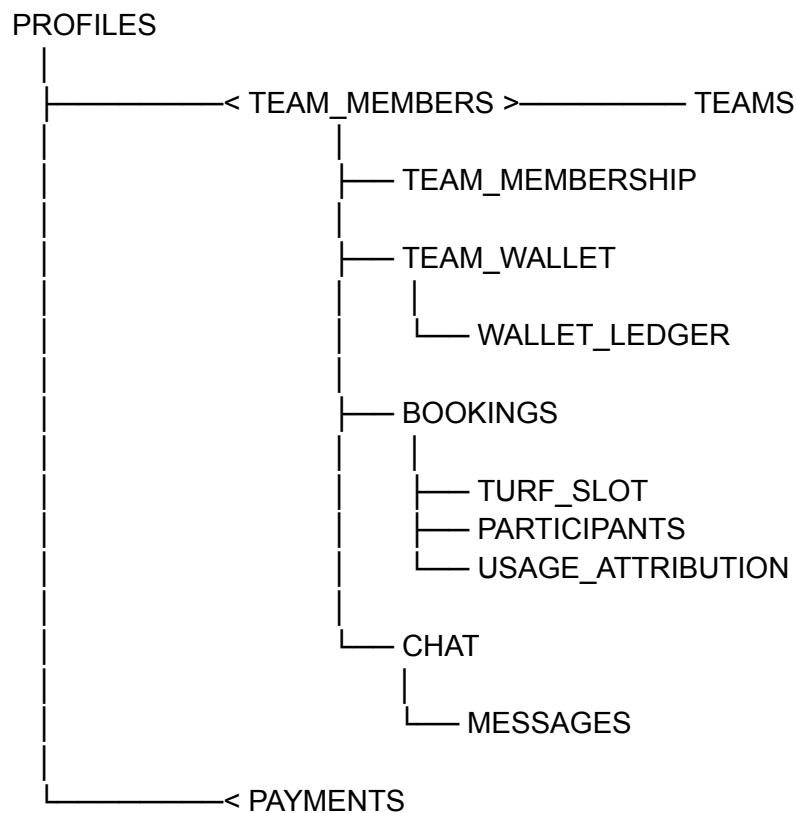
MEMBER  
Rank 1  
Member A  
2,400 credits used

The underlying booking and usage records remain authoritative; this table is only a convenient weekly representation.

---

## 6.27 Main Relationships

The final ER relationship can be represented as:

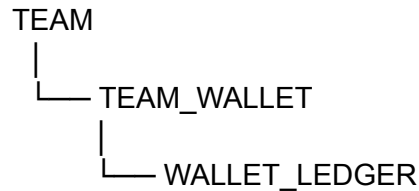


This follows the source blueprint's core relationship model while applying the final Team terminology.

---

## 6.28 Credit Ownership Relationship

The most important financial relationship is:



Not:



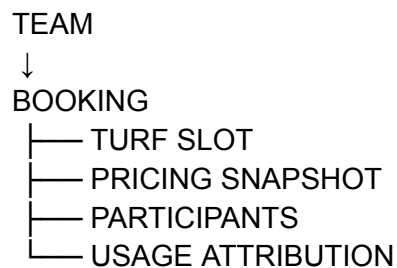
There are no individual Team-credit wallets in the MVP.

Member usage is an analytics record only.

---

## 6.29 Booking Relationship

Every booking should be traceable through:

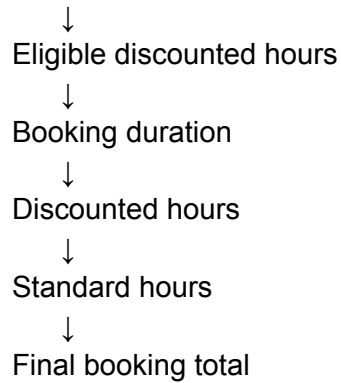


This gives the system enough historical information to explain:

Why did this booking cost 1,800 credits?

The answer should be reconstructable from:

Active Team Plan



The source blueprint specifically requires enough stored pricing data to reconstruct the booking cost.

---

## 6.30 Rolling 24-Hour Pricing Data

For the ₹10,000 membership, the database must make it possible to determine:



Example:

Allowance = 3 hours

Previous eligible usage = 2 hours

New booking = 2 hours



Remaining discounted = 1 hour

Therefore:

$1 \times ₹800$

$1 \times ₹1,000$

Total = ₹1,800

The final price must be calculated by the trusted backend and saved as the booking's historical pricing snapshot.

---

## 6.31 Recommended Indexes

Indexes should be based on actual application query patterns.

Initial important indexes:

`team_members(team_id, user_id)`

`team_members(user_id, membership_status)`

`bookings(team_id, status)`

`bookings(slot_id, status)`

`bookings(start_time)`

`bookings(created_at)`

`booking_participants(booking_id)`

`booking_participants(user_id)`

`wallet_ledger(team_id, created_at)`

`payments(team_id, status)`

`chat_messages(team_id, created_at)`

notifications(user\_id, read\_at)

member\_usage\_attribution(user\_id, created\_at)

leaderboard\_weekly(period\_start, leaderboard\_type)

The source blueprint recommends these same query-oriented indexes and notes that the final index set should be validated against actual production query patterns.

---

## 6.32 Unique Constraints

The database should enforce business-critical uniqueness.

### **Team membership**

UNIQUE(team\_id, user\_id)

### **Booking reference**

UNIQUE(booking\_reference)

### **Team code**

UNIQUE(team\_code)

### **Payment verification / activation**

Instead of gateway IDs, the final implementation should enforce a suitable unique relationship preventing the same verified membership payment from generating multiple credit allocations.

For example, the activation transaction should be idempotent around:

membership\_id  
+  
verified payment  
+  
credit allocation

The source blueprint emphasizes uniqueness and idempotency for memberships, bookings and payment-driven credit allocation.

---

## 6.33 Foreign-Key Integrity

Database relationships should use foreign keys.

Examples:

```
bookings.team_id
    ↓
teams.id
booking_participants.booking_id
    ↓
bookings.id
booking_participants.user_id
    ↓
profiles.id
wallet_ledger.team_id
    ↓
teams.id
```

This prevents orphaned financial, booking and participant records. The source blueprint explicitly requires foreign-key integrity for this purpose.

---

## 6.34 Soft Delete vs Hard Delete

Critical historical records must not be hard-deleted.

Do not hard-delete:

```
wallet_ledger
payments
completed bookings
completed participation
admin_audit_logs
```

Use states such as:

```
CANCELLED
REMOVED
ARCHIVED
```

Temporary records such as expired slot holds may eventually be cleaned up after an appropriate retention period.

This preserves the historical record required for financial and operational auditing.

---

## 6.35 Database Transaction Boundaries

The most important workflows must be atomic.

### Booking confirmation

Check hold  
+  
Check availability  
+  
Check credits  
+  
Calculate price  
+  
Deduct Team wallet  
+  
Create ledger entry  
+  
Confirm booking

Either all succeed or none succeed.

### Eligible cancellation

Validate cancellation  
+  
Cancel booking  
+  
Refund Team credits  
+  
Create ledger entry  
+  
Restore discounted-hour usage

### Membership activation

Verify external payment  
+  
Record payment as VERIFIED  
+  
Activate membership  
+  
Allocate Team credits  
+  
Create ledger entry  
+  
Create audit record

The source blueprint explicitly defines these transaction boundaries and the rule that required changes must either all succeed or none do.

---

## 6.36 Source of Truth

The final source-of-truth map is:

| Data                   | Source of Truth                                  |
|------------------------|--------------------------------------------------|
| User identity          | Supabase Auth                                    |
| User profile           | <code>profiles</code>                            |
| Team membership        | <code>team_members</code>                        |
| Team role              | <code>team_members</code>                        |
| Active membership plan | <code>team_memberships</code>                    |
| Current credits        | <code>team_wallets</code>                        |
| Credit history         | <code>wallet_ledger</code>                       |
| External payment state | <code>payments</code>                            |
| Slot availability      | Turf inventory + holds + bookings + Admin blocks |
| Booking state          | <code>bookings</code>                            |
| Participants           | <code>booking_participants</code>                |

|               |                                                                     |
|---------------|---------------------------------------------------------------------|
| Member usage  | <code>member_usage_attribution</code>                               |
| Chat          | <code>chat_messages</code>                                          |
| Notifications | <code>notifications</code>                                          |
| Admin actions | <code>admin_audit_logs</code>                                       |
| Leaderboard   | Derived from completed records /<br><code>leaderboard_weekly</code> |

This follows the source blueprint's source-of-truth architecture, with the payment row updated for the final external-verification model.

---

## 6.37 What Must NOT Become Independent Truth

Do not duplicate the same business value across unrelated tables.

Bad:

`team_wallets.available_credits = 10,000`

`teams.credit_balance = 10,000`

`profiles.team_credit_balance = 10,000`

These can eventually disagree.

Prefer:

`team_wallets.available_credits`

as the current balance source.

Likewise, do not store manually editable leaderboard totals as the only source of truth.

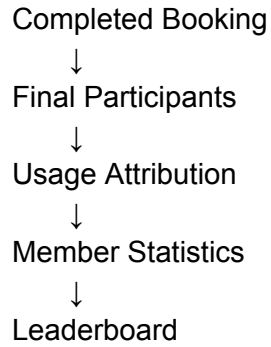
The source blueprint explicitly recommends operational records rather than duplicate manually maintained totals.

---

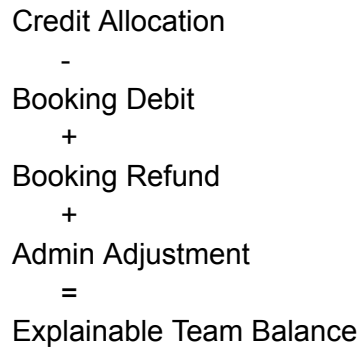
## 6.38 Analytics as Derived Data

Most analytics should be generated from operational records.

For member usage:



For Team credits:

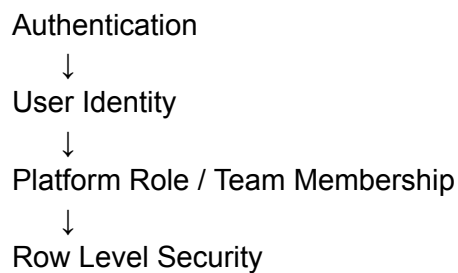


This makes analytics auditable and avoids manual statistics maintenance.

---

## 6.39 Database Security Boundary

Database access should follow:



↓  
Server-Side Validation  
↓  
Database Transaction

Being authenticated does **not** automatically mean being authorized to perform an operation.

For example:

Authenticated Member  
+  
Not Host/Co-host  
↓  
Cannot initiate Team booking

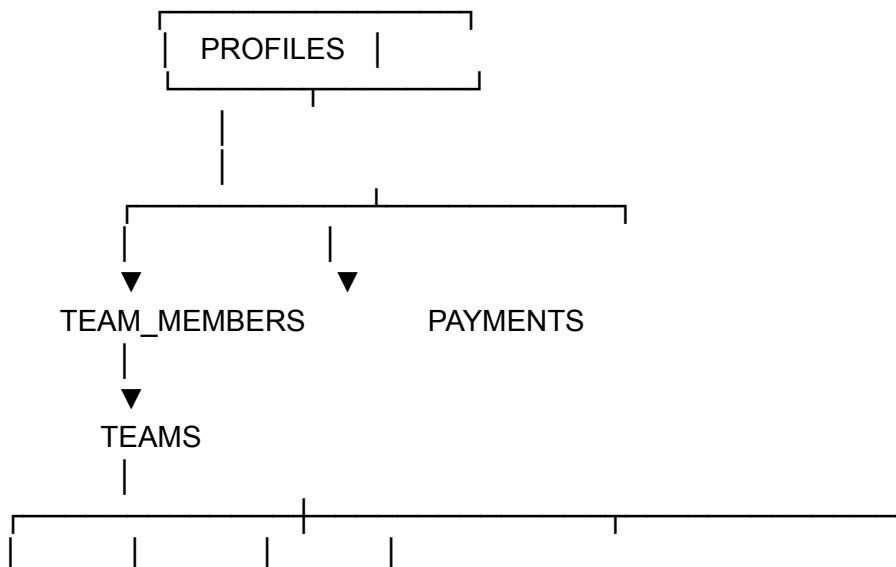
and:

Authenticated User  
+  
Not Admin  
↓  
Cannot block Turf globally

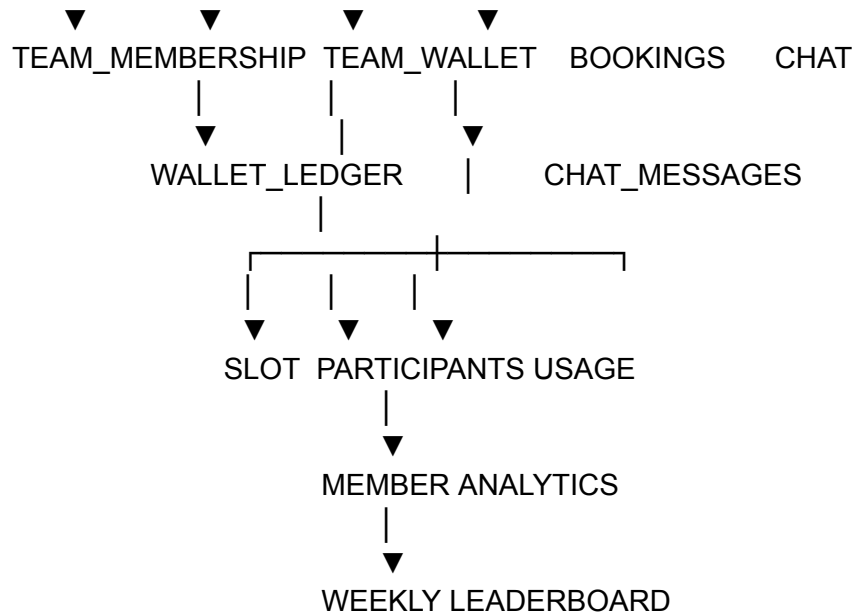
The source blueprint defines this layered database security boundary explicitly.

---

## 6.40 Simplified Final ER Model







This is the simplified structure derived from the source blueprint's ER model.

---

## 6.41 Final Database Principle

The database should record **what actually happened**, not merely the latest totals displayed by the UI.

For example, instead of:

Member A  
credits\_used = 2,400

store:

Booking 1 → Member A → 300  
 Booking 2 → Member A → 500  
 Booking 3 → Member A → 400  
 ...

Then:

SUM(completed usage)  
 ↓  
 2,400

Likewise:

Team Credits

=

Allocations

- Booking Debits

+ Refunds

+ Admin Adjustments

This operational-event approach makes the system explainable, auditable and much safer for the credit/booking model.

**Final core chain:**

TEAM



MEMBERSHIP



WALLET



LEDGER



BOOKING



PARTICIPANTS



SESSION COMPLETION



USAGE ATTRIBUTION



ANALYTICS



LEADERBOARDS

## 8. CLIENT vs SERVER RESPONSIBILITY

This section defines exactly what belongs inside the **React Native + Expo mobile application** and what must be controlled by the **Supabase/backend layer**.

The central rule is:

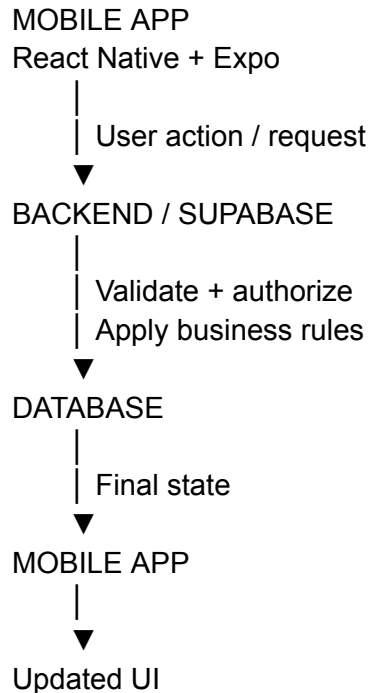
**The mobile app requests and displays. The backend validates, decides and commits. The database records the final truth.**

Any operation that can affect **credits, payments, bookings, permissions or historical records** must not depend on client-side logic alone.

---

## 8.1 Responsibility Boundary

The overall boundary is:



The client can suggest or preview an operation, but the backend determines whether that operation is actually valid.

---

## 8.2 What the Mobile App Is Responsible For

The mobile application handles the **presentation and interaction layer**.

### UI

Screens  
Navigation

- Forms
- Inputs
- Buttons
- Loading states
- Error states
- Success states
- Dialogs
- Modals
- Timers
- Countdowns
- Data presentation

### **User interaction**

- Select Team
- Select Turf
- Select Date
- Select Time
- Select Participants
- Enter Team ID
- Enter chat message
- Choose membership plan
- Request action

### **Local presentation**

The app may display:

- Team credit balance
- Available Turf slots
- Booking price preview
- Booking status
- Participant list
- Usage insights
- Notifications
- Chat
- Leaderboards

But these values must ultimately come from authoritative backend data. The source blueprint makes the same distinction between mobile presentation and backend authority.

---

## **8.3 What the Mobile App Must NOT Decide**

The mobile application must never be the final authority for:

- Whether payment is verified
- Whether membership is active
- Whether credits were allocated
- Whether Team has sufficient credits
- Whether Turf slot is available
- Whether a hold is still valid
- Final booking price
- Whether booking is confirmed
- Whether cancellation is eligible
- Refund amount
- Discounted-hour usage
- Final participant lock
- Member usage attribution
- Leaderboard score
- Admin authorization

For example, the app may calculate:

2 hours × ₹800  
= ₹1,600

as a **preview**.

The backend must still recalculate the actual price using current membership and rolling-24-hour usage before confirmation.

---

## 8.4 Authentication Responsibility

### Mobile App

The app:

Displays Login



User selects authentication method



Receives authenticated session



Uses session for API/database requests

## Backend

The backend:

Validate authenticated session



Identify actual authenticated user



Load profile



Resolve platform role



Resolve Team memberships

The backend must not blindly trust a `user_id` sent by the client.

For example:

Client sends:

`user_id = 123`

The backend should instead derive the actual user from the authenticated session. This is a core security boundary in the source blueprint.

---

## 8.5 Role and Permission Responsibility

The mobile app can use roles to decide **what UI to show**.

For example:

Platform Role = MEMBER

Show:

Home

Teams

Book

Activity

Profile

A Host may see:

Team Management  
Wallet  
Bookings  
Participants

An Admin may see:

Admin Home  
Teams  
Bookings  
Memberships  
Credits  
Turf Management  
Analytics

However, hiding a button is **not security**.

The backend must independently validate:

Who is requesting?



What is their platform role?



What is their Team role?



Does that role permit this operation?

The source blueprint explicitly establishes authorization as a backend responsibility rather than something determined only by the client UI.

---

## 8.6 Team Isolation

The client may send:

`team_id = TEAM_A`

but the backend must verify that the user is actually authorized to access Team A.

Example:

User



Member of Team A

↓  
Request Team A data  
↓  
ALLOW

But:

User  
↓  
Not member of Team B  
↓  
Request Team B data  
↓  
REJECT

Changing `team_id` in a network request must never allow a user to access another Team.

This should be enforced through authentication, authorization, RLS and server-side validation.

---

## 8.7 Membership Plan Responsibility

### Mobile App

The app may:

- Display ₹10,000 plan
- Display ₹25,000 plan
- Display plan benefits
- Allow plan selection
- Show estimated pricing
- Show membership status

### Backend

The backend must determine:

- Which plan actually exists
- +
- Current plan configuration
- +
- Team membership state
- +



Discount rules  
+  
Credit allocation

The client must not be able to submit:

membership\_rate = 800  
discounted\_hours = 999  
credit\_allocation = 100000

and have the server accept those values blindly.

---

## 8.8 External Payment Responsibility

The finalized MVP uses **external payment + Admin verification**.

Therefore:

### Mobile App

The mobile app may:

- Show selected membership
- Show amount
- Show payment instructions
- Display payment status
- Allow external-reference entry where applicable
- Show "Pending Admin Verification"

The app does **not** verify the payment.

### Admin

Admin:

- Reviews payment
- Contacts Host / Co-host where needed
- Checks external payment
- Enters/records required reference
- Marks payment VERIFIED or REJECTED

### Backend

The backend must:

- Validate Admin permission
- Validate payment state
- Prevent duplicate verification/allocation
- Record payment
- Activate membership
- Allocate Team credits
- Create wallet ledger
- Create audit record

The previous gateway/webhook responsibilities in the old blueprint are therefore replaced by the **Admin verification workflow** in this final MVP.

---

## 8.9 Wallet Responsibility

The mobile app may display:

Team Credits  
8,200

But it must never execute:

balance = balance - 800

Instead:

Confirm Booking  
↓  
Backend Transaction  
↓  
Deduct Team Wallet  
↓  
Create Ledger  
↓  
Return Updated Balance

This is one of the most important boundaries in the system because wallet data represents actual Team credit state.

---

## 8.10 Slot Availability Responsibility

### Mobile App

The app:

Requests availability

Displays slots

Allows user to select a slot

### Backend

The backend:

Checks Turf resource

+

Checks slot

+

Checks Admin block

+

Checks active holds

+

Checks confirmed bookings

The mobile app must never assume:

"It looked available a second ago, so it is still available."

The backend decides the actual state at the moment of the operation.

---

## 8.11 One-Minute Hold Responsibility

### Mobile App

The app:

Starts visual countdown

Shows:

00:59

00:58

00:57

...

## Backend

The backend owns:

hold\_created\_at  
hold\_expires\_at  
current server time  
hold status

The actual rule is:

$\text{NOW} < \text{expires\_at}$

↓

Hold valid

The client timer is only visual.

The source blueprint explicitly requires the backend to determine hold validity rather than trusting the client countdown.

---

## 8.12 Price Calculation Responsibility

This is a **server-authoritative** calculation.

### Mobile App

Can calculate a preview:

Estimated price = ₹1,600

### Backend

Must recalculate:

Active Membership

↓

Membership rate

↓

Standard rate

↓  
Rolling 24-hour discounted usage  
↓  
Remaining allowance  
↓  
Booking duration  
↓  
Discounted hours  
↓  
Standard hours  
↓  
FINAL TOTAL

For example:

₹10,000 plan

Previous usage = 2 hours

New booking = 2 hours

1 hour × ₹800

1 hour × ₹1,000

Final = ₹1,800

The backend result is the only result that can be used for the actual booking transaction.

---

## 8.13 Booking Confirmation Responsibility

The client sends something conceptually like:

Confirm Booking

booking\_attempt\_id = XYZ

The backend then checks:

Hold valid?

+

Slot available?

+

Team membership active?

+

Team credits sufficient?

+  
Price valid?  
+  
Requesting user authorized?  
+  
Participants valid?

Only after all checks pass:

Deduct Team credits  
↓  
Create wallet ledger  
↓  
Confirm booking  
↓  
Commit transaction

This validation sequence is explicitly defined in the source blueprint.

---

## 8.14 Credit Deduction Responsibility

Client:

REQUEST ONLY

Backend:

Validate balance  
↓  
Lock / transact wallet  
↓  
Deduct credits  
↓  
Create ledger entry  
↓  
Return updated balance

The client never receives permission to directly mutate:

available\_credits  
reserved\_credits

---

# 8.15 Cancellation Responsibility

## Mobile App

The app can:

Show Cancel button  
Show cancellation policy  
Send cancellation request  
Display result

## Backend

The backend determines:

Booking state  
+  
Current server time  
+  
Scheduled start  
+  
24-hour rule  
+  
Refund eligibility

Then:

Eligible  
↓  
Cancel  
↓  
Refund credits  
↓  
Create ledger  
↓  
Restore qualifying discounted usage

or:

Late cancellation  
↓  
Cancel  
↓  
No credit refund

The source blueprint requires cancellation eligibility and refund processing to be handled on the backend.

---

## 8.16 Participant Responsibility

### Mobile App

The app allows:

- Select participants
- Add participants
- Remove participants
- Display participant list

### Backend

The backend validates:

- Is user an eligible Team member?
- +
- Is booking still editable?
- +
- Has session started?

Before the session:

Participants editable

At session start:

Participants locked

After completion:

Final participant list becomes historical

The source blueprint explicitly separates Team membership from booking participants and requires final participant locking.

---



## 8.17 Usage Attribution Responsibility

The client must not calculate authoritative historical member usage.

Backend:

Completed Booking



Final Participants



Actual Booking Credits



Credits ÷ Final Participant Count



Usage Attribution



Analytics

Example:

Booking = 1,800 credits

Participants = 6

Usage = 300/member

The client only displays the resulting usage data.

The source blueprint explicitly assigns usage attribution to the backend and treats it as analytics rather than a credit transfer.

---

## 8.18 Leaderboard Responsibility

The mobile app:

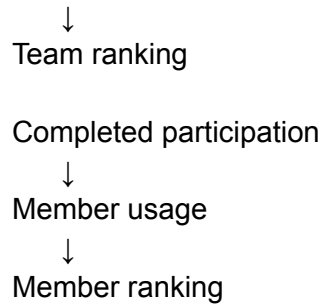
Requests leaderboard



Displays rankings

Backend:

Completed bookings



The client must never submit or directly modify leaderboard values.

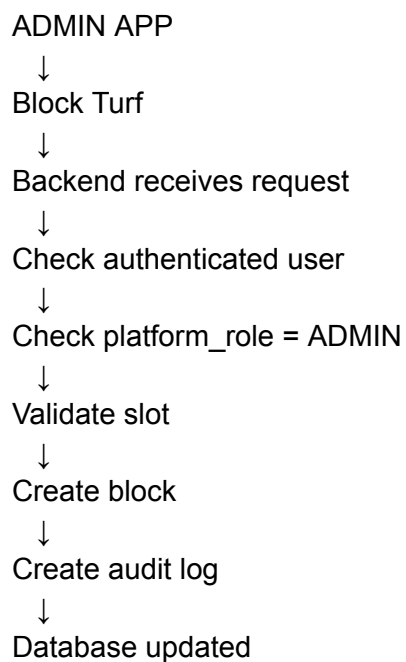
---

## 8.19 Admin Operation Responsibility

Admin UI is still client-side UI within the same React Native app.

But every sensitive Admin operation must be server-authorized.

Example:



This applies to:

Membership verification  
Credit allocation  
Credit adjustment

- Turf blocking
- Turf unblocking
- Admin booking
- Booking cancellation
- Role changes
- Member removal

The source blueprint requires sensitive Admin actions affecting credits or booking history to be auditable.

---

## 8.20 Chat Responsibility

### Mobile App

- Compose message
- Send message
- Display messages

### Backend

- Authenticate sender
- +
- Validate Team membership
- +
- Validate chat access
- +
- Apply moderation rules
- +
- Store message

Realtime may then deliver the stored message to other authorized Team members.

---

## 8.21 Notifications Responsibility

### Backend

The backend determines that an event happened:

- Booking confirmed

Booking cancelled  
Membership activated  
Admin booking created  
Chat activity

Then:

Create notification record  
+  
Trigger push notification

## Mobile App

The app:

Displays notification  
Updates notification state

A notification is never proof that an event occurred.

The database record is the source of truth.

---

## 8.22 Client vs Server Decision Matrix

| Operation            | Mobile App | Backend / DB           |
|----------------------|------------|------------------------|
| Login UI             | ✓          | ✓ Validate             |
| Display profile      | ✓          | ✓ Source               |
| Display Teams        | ✓          | ✓ Source               |
| Select Team          | ✓          | ✓ Validate access      |
| Select Turf          | ✓          | ✓ Validate             |
| Display availability | ✓          | ✓ Source               |
| Create 1-minute hold | ✗          | ✓                      |
| Visual countdown     | ✓          | ✓ Authoritative expiry |

|                          |         |                     |
|--------------------------|---------|---------------------|
| Price preview            | ✓       | ✓ Final calculation |
| Rolling 24h calculation  | ✗       | ✓                   |
| Confirm booking          | Request | ✓                   |
| Deduct credits           | ✗       | ✓                   |
| Create ledger            | ✗       | ✓                   |
| Verify external payment  | ✗       | ✓ Admin/backend     |
| Activate membership      | ✗       | ✓                   |
| Allocate credits         | ✗       | ✓                   |
| Cancel booking           | Request | ✓                   |
| Refund credits           | ✗       | ✓                   |
| Restore discounted hours | ✗       | ✓                   |
| Add participants         | Request | ✓ Validate          |
| Lock participants        | ✗       | ✓                   |
| Usage attribution        | ✗       | ✓                   |
| Leaderboard calculation  | ✗       | ✓                   |
| Send chat                | Request | ✓ Validate/store    |
| Read chat                | ✓       | ✓ Authorize         |
| Block Turf               | ✗       | ✓ Admin only        |
| Credit adjustment        | ✗       | ✓ Admin only        |
| Role change              | ✗       | ✓ Authorized        |
| Audit logging            | ✗       | ✓                   |

This matrix follows the responsibility boundary established in the source architecture, with payment responsibilities updated to the finalized external-verification model.

---

## 8.23 What Can Be Calculated Locally

Client-side calculations are allowed when they exist purely for **user experience**.

Examples:

Countdown display  
UI totals  
Formatted currency  
Progress indicators  
Simple presentation calculations

But:

Preview  $\neq$  Final Business Decision

For example:

CLIENT

2 hours selected  
↓  
"Estimated ₹1,600"

Then:

BACKEND

Active plan  
+  
Rolling usage  
+  
Current pricing  
+  
Booking duration  
↓  
Final = ₹1,800

The backend value wins.

---

## 8.24 Backend Error Contract

The backend should return structured business outcomes rather than exposing raw database errors.

Examples:

BOOKING\_CONFIRMED  
INSUFFICIENT\_CREDITS  
SLOT\_UNAVAILABLE  
HOLD\_EXPIRED  
MEMBERSHIP\_INACTIVE  
CANCELLATION\_NOT\_ALLOWED  
PAYMENT\_PENDING  
PAYMENT\_REJECTED  
TEAM\_NOT\_FOUND  
UNAUTHORIZED  
FORBIDDEN

The mobile app converts these into human-readable UI.

For example:

Backend:  
HOLD\_EXPIRED

UI:  
"Your slot hold has expired. Please select the slot again."

The source blueprint specifies this structured error boundary and says technical server/database errors should not be exposed directly to users.

---

## 8.25 Atomic Operation Responsibility

Whenever an operation touches multiple critical records, the backend must perform the operation atomically.

### Booking

Check Hold  
+  
Check Slot

- +
- Check Credits
- +
- Calculate Price
- +
- Deduct Wallet
- +
- Create Ledger
- +
- Confirm Booking

### **Cancellation**

- Validate Cancellation
- +
- Cancel Booking
- +
- Refund Wallet
- +
- Create Ledger
- +
- Restore Discounted Usage

### **Membership Activation**

- Verify Payment
- +
- Record Payment
- +
- Activate Membership
- +
- Allocate Credits
- +
- Create Ledger
- +
- Create Audit Log

The database should reach either:

ALL CHANGES SUCCESSFUL

or:

NO PARTIAL FINANCIAL / BOOKING STATE

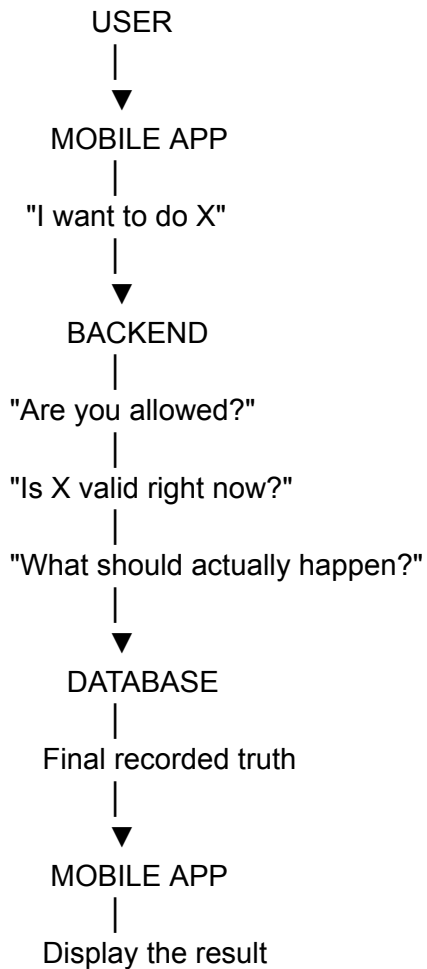
This atomicity requirement is explicitly defined in the database design.



---

## 8.26 Final Client/Server Rule

The complete responsibility model is:



So the fundamental implementation rule for the entire MVP is:

**CLIENT**

= Interaction + Presentation

**BACKEND**

= Authorization + Business Logic + Validation

**DATABASE**

= Persistent Source of Truth

## LEDGER / AUDIT

= Financial + Administrative History

This is the boundary that should be followed consistently when implementing the application with an AI coding agent or manually. The source blueprint likewise states that the client decides what the user wants to do, the backend decides whether it is allowed and what happens, and the database records the final truth.

# 9. SECURITY RULES

This section defines the security requirements for the **Thrill Mill Club MVP**.

Security is a core system requirement because the application handles **real membership payments, Team credits, Turf inventory, bookings, member information and privileged Admin operations**. The source blueprint identifies server-side authorization, protected Admin actions, auditability, double-booking prevention and duplicate-credit prevention as minimum security requirements.

The final security principle is:

AUTHENTICATE



AUTHORIZE



VALIDATE



EXECUTE



RECORD

No sensitive operation should bypass this sequence.

---

## 9.1 Authentication Security

Every protected operation must originate from an authenticated user session.

The system must:

User



Supabase Auth



Authenticated Session



Identify User



Load Profile

The backend must determine the authenticated user from the trusted session.

It must **not** blindly trust:

`user_id`

supplied in the request body.

Example:

Client:

`user_id = 123`

The backend instead uses:

Authenticated Session



Actual Authenticated User

The source blueprint explicitly requires server-side session validation and warns against trusting client-supplied user identity.

---

## 9.2 Authorization Security

Authentication answers:

"Who is this user?"

Authorization answers:

"What is this user allowed to do?"

Both must be checked.

Example:

User  
↓  
Authenticated  
↓  
Team A  
↓  
Role = MEMBER  
↓  
Attempt to block Turf  
↓  
REJECT

Whereas:

Admin  
↓  
Authenticated  
↓  
Platform Role = ADMIN  
↓  
Block Turf  
↓  
ALLOW

Hiding an Admin button in the mobile UI is not a security control. The backend must independently enforce authorization.

---

## 9.3 Team Data Isolation

A user must only access Team data for Teams where they have the appropriate authorization.

For example:

Team A  
Members:  
A, B, C

Member A must not be able to access Team B simply by changing:

team\_id = Team\_B

in an API/database request.

The system must enforce Team isolation through:

Authentication

+

Authorization

+

RLS

+

Server-side validation

The source blueprint explicitly calls out this attack scenario and requires authorization at the database/server layer.

---

## 9.4 Team Role Isolation

Team roles are contextual.

Example:

User A

Team A → HOST

Team B → MEMBER

Team C → CO\_HOST

Therefore:

HOST privilege in Team A

≠

HOST privilege in Team B

The backend must check the user's role against the **specific Team involved in the operation**.

A Host of Team A must not receive Host permissions for Team B merely because they are a Host somewhere else.

---

## 9.5 Admin Security

Admin is a **platform-level** role.

Admin-only operations include:

- Block Turf
- Unblock Turf
- Perform Admin booking
- Adjust Team credits
- Verify external payment
- Activate membership
- Change roles
- Remove members
- Access global analytics
- Perform permitted booking overrides

Every such operation must be checked server-side:

Authenticated User

↓  
platform\_role = ADMIN?  
↓  
YES → Continue  
NO → Reject

The source blueprint explicitly states that Admin actions must be protected at the backend and that UI-level hiding is insufficient.

---

## 9.6 Row Level Security

Supabase Row Level Security should be enabled for tables where client-accessible operations require row-level authorization.

The security question for every table is:

USER  
↓  
What records can this user READ?  
↓  
What records can this user CREATE?  
↓  
What records can this user MODIFY?

RLS policies should be derived from the Section 2 permission matrix.

Conceptually:

Authenticated User



Team Membership?



Team Role?



Allowed operation?



RLS policy

The source blueprint explicitly identifies RLS as a security layer between authentication and server-side business validation.

---

## 9.7 Sensitive Operations Must Use Trusted Server Logic

The following must **not** be implemented as unrestricted client-side database updates:

Team wallet balance

Credit ledger

Payment status

Membership activation

Booking confirmation

Refund

Slot hold

Final pricing

Admin credit adjustment

Leaderboard totals

These require trusted backend/database logic.

The client can request:

"Book this slot"

but cannot directly execute:

wallet -= 1800

The source blueprint explicitly identifies these operations as server-controlled.

---

## 9.8 Wallet Security

The following wallet rules are mandatory.

### **Rule 1 — No direct member modification**

Members cannot directly modify Team credit balances.

### **Rule 2 — Every credit movement has a ledger entry**

Credit balance change

↓

Wallet ledger record

### **Rule 3 — Wallet cannot become negative**

`available_credits >= 0`

### **Rule 4 — Booking cannot exceed available credits**

`Booking Cost <= Available Team Credits`

### **Rule 5 — Duplicate requests cannot duplicate deductions**

For example:

Confirm Booking

↓

Request #1

↓

-1800

A repeated request for the same booking must not produce:

Request #2

↓

-1800 AGAIN

### **Rule 6 — Financial history remains auditable**

The source blueprint defines all six of these wallet requirements.

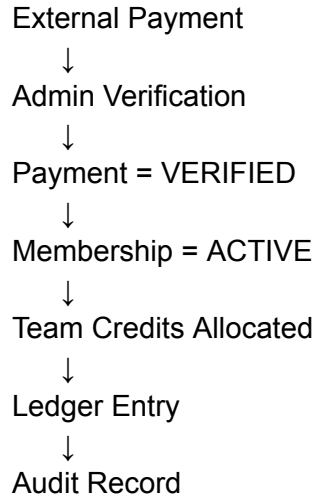


---

## 9.9 External Payment Security

The final MVP uses **external payment + Admin verification**, not an in-app payment gateway.

Therefore the final flow is:



The security rule remains the same as the original blueprint:

A client-side indication of payment success must never by itself allocate credits.

Under the final MVP, there is no Razorpay dependency; the Admin verification record becomes the trusted payment confirmation step.

---

## 9.10 Payment / Membership Idempotency

The same verified payment or activation action must not allocate credits twice.

Example:

Admin verifies payment  
↓  
+10,000 credits

Repeated activation:

Same membership



Already activated



NO additional credits

The final implementation should therefore maintain a unique/idempotent relationship between:

Membership

+

Verified Payment

+

Credit Allocation

The original blueprint's underlying requirement is that one successful payment must result in one credit allocation, even when retries/events are repeated.

---

## 9.11 Booking Security

The booking system must prevent:

Double booking

Expired-hold confirmation

Booking a blocked slot

Booking without sufficient credits

Unauthorized booking

Duplicate confirmation

Manipulated booking price

All of these must be checked by the backend.

The source blueprint explicitly identifies these as required booking protections.

---

## 9.12 Slot-Hold Security

The one-minute hold must use a server-controlled expiration timestamp.

Example:

created\_at = 18:30:00  
expires\_at = 18:31:00

The client must not be allowed to:

change expires\_at  
extend the hold  
change the local clock

The backend decides whether:

current\_server\_time < expires\_at

The mobile countdown is purely visual.

---

## 9.13 Pricing Security

The client may request a price preview but cannot determine the final booking price.

Example attack:

Actual price = ₹1,000/hour

Malicious client sends:  
price = ₹1

The backend must ignore the client-provided final price and calculate independently.

The calculation must use:

Active Team plan  
+  
Booking duration  
+  
Rolling 24-hour usage  
+  
Remaining discounted allowance  
+  
Standard rate  
+  
Current configured pricing

The source blueprint explicitly defines this protection.

---

## 9.14 ₹10,000 Plan Abuse Prevention

The three discounted hours must be calculated from trusted backend records.

The client cannot claim:

discounted\_hours\_used = 0

to obtain an incorrect discount.

The backend calculates:

Qualifying booking history



Relevant timestamps



Rolling 24-hour window



Discounted hours already used



Remaining allowance

Therefore the client cannot manipulate the 3-hour allowance through request parameters.

---

## 9.15 Cancellation Security

Cancellation eligibility must be determined using **server time**.

The client cannot decide:

24 hours remaining = YES

The backend calculates:

Booking Start Time

-

Current Server Time

=

Remaining Time

Then applies the cancellation rule.

For an eligible cancellation:

Cancel  
↓  
Refund  
↓  
Ledger  
↓  
Restore qualifying discounted usage

For a late cancellation:

Cancel  
↓  
No credit refund

The source blueprint explicitly assigns this timing decision to the backend.

---

## 9.16 Refund Security

A refund must be tied to the specific cancelled booking.

The system must prevent:

Booking ABC  
↓  
Refund #1 → +800  
Refund #2 → +800

Only one valid refund operation may affect the wallet.

The refund ledger entry should reference the booking/refund event.

This prevents retrying a cancellation request from generating multiple wallet credits.

---

## 9.17 Participant Security

Only authorized users can modify booking participants.

The backend must verify:

Authenticated User



Active Team Membership



Booking belongs to same Team



User has appropriate role



Booking is still editable

A user must not be able to add themselves to another Team's booking by submitting:

`booking_id = ABC`

The source blueprint explicitly defines this association check.

---

## 9.18 Historical Record Protection

The following records should not be freely edited or deleted:

Payments

Wallet ledger

Completed bookings

Completed participation

Admin audit logs

When a correction is required, prefer:

Original Event



Correction / Adjustment Event

rather than silently rewriting the original historical record.

This preserves auditability and makes the credit system explainable.

---

## 9.19 Admin Credit Adjustment Security

Every manual Admin credit adjustment must contain:

Team  
Amount  
Reason  
Admin identity  
Timestamp  
Reference

Example:

ADMIN\_ADJUSTMENT

Team: Team A  
Amount: +500  
Reason: Promotional credit  
Admin: Admin User  
Timestamp: 2026-08-28

The adjustment must produce:

Wallet change  
+  
Ledger entry  
+  
Audit record

This allows investigation of unusual credit movements.

---

## 9.20 Chat Security

Team chat must enforce:

Authenticated access  
+  
Active Team membership  
+  
Team-specific access  
+

Server-side validation

+

Basic prohibited-content filtering

+

Moderation controls

A user must not be able to read another Team's chat by changing:

team\_id

or:

chat\_room\_id

in a request.

The source blueprint explicitly requires Team-level chat isolation.

---

## 9.21 Push Notification Security

Notifications should avoid exposing unnecessary sensitive information on lock screens.

Prefer:

"Your Turf booking has been updated."

rather than exposing detailed financial or private information.

The user can open the authenticated application to retrieve the full information.

This notification-minimization rule is explicitly included in the source blueprint.

---

## 9.22 Secrets Management

Privileged credentials must never be included in:

React Native source code

Mobile bundle

Client configuration



Public Git repository

For the final MVP, this includes:

- Supabase service-role key
- Server-side notification credentials
- Any external payment/admin credentials
- Other privileged API secrets

They must remain in server-side environment/secret configuration.

The original blueprint also lists Razorpay secret/webhook credentials; those specifically become irrelevant because Razorpay has been removed from the final MVP. The general rule—**privileged credentials never reach the client**—remains.

---

## 9.23 Input Validation

Every externally supplied value must be validated.

Examples:

- Booking duration
- Credit amount
- Team ID
- Booking ID
- Slot ID
- Participant ID
- Message content
- Payment reference
- Membership ID

Validation should include:

- Data type
- +
- Allowed range
- +
- Ownership / association
- +
- Current state
- +
- Permission

+  
Business rules

For example:

`credit_amount = -999999`

must not automatically become a valid Admin adjustment.

The source blueprint explicitly defines these validation dimensions.

---

## 9.24 Rate Limiting / Abuse Protection

Sensitive operations should have reasonable protection against excessive requests.

Particularly:

Authentication  
Payment verification  
Booking creation  
Booking confirmation  
Chat messages  
Admin operations

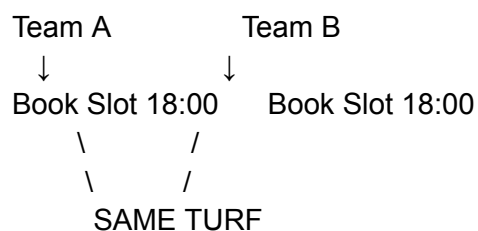
The MVP does not require a sophisticated anti-abuse platform, but obvious request abuse should not be completely unprotected.

---

## 9.25 Concurrency Security

The system must handle simultaneous operations safely.

Example:



↓  
Only one succeeds

Other examples:

Two confirmation requests

↓  
Same booking

or:

Two credit deductions

↓  
Same Team

Database transactions, constraints and appropriate locking must protect these operations.

---

## 9.26 Error Disclosure

User-facing errors should contain only information useful to the user.

Example:

User sees:

"This Turf slot is no longer available."

Internal logs can contain:

Booking conflict

Slot ID

Team ID

Request ID

Timestamp

The application should not expose:

Database structure

Internal query errors

Server stack traces

Secret/configuration details

Infrastructure details

The source blueprint explicitly establishes this separation.

---

## 9.27 Auditability

Security-sensitive actions must be traceable.

Examples:

Credit adjustment  
Role change  
Booking override  
Cancellation override  
Slot blocking  
Refund  
Payment verification  
Membership activation

The audit trail should allow the club to answer:

WHO

↓

did WHAT

↓

to WHICH resource

↓

WHEN

↓

and WHY?

This is a core requirement for Admin operations.

---

## 9.28 Security Priority

For the MVP, security priorities are:

1. Payment / Membership protection

↓

2. Credit / Wallet integrity

↓

3. Booking / Slot integrity

- ↓  
4. Authorization  
↓  
5. Team data isolation  
↓  
6. Auditability  
↓  
7. Chat / Content protection  
↓  
8. General abuse protection

This ordering follows the source blueprint's security priority.

---

## 9.29 Critical Security Invariants

The system must continuously preserve these invariants:

### WALLET

Team credits cannot become negative.

### BOOKING

One Turf slot cannot have two confirmed bookings.

### PAYMENT

One verified membership payment cannot allocate credits twice.

### LEDGER

Every credit balance change has a corresponding ledger event.

### PARTICIPANT

Only eligible Team members can be participants.

### CANCELLATION

Refund only when cancellation policy permits it.

### USAGE

Usage is based on actual completed booking credits.

### ROLE

Team role applies only to that Team.

## ADMIN

Admin operations require platform-level Admin authority.

## HOLD

Expired holds cannot become confirmed bookings.

## PRICING

Client cannot determine final price.

These invariants are central to the source blueprint's data-integrity model.

---

# 9.30 Production Security Validation

Before production release, the following must be tested explicitly:

- ✓ Member cannot modify Team wallet
- ✓ Member cannot access another Team's data
- ✓ Member cannot access Admin functions
- ✓ Host permissions remain Team-specific
- ✓ Expired hold cannot be confirmed
- ✓ Same slot cannot be booked twice
- ✓ Verified payment cannot create duplicate credits
- ✓ Refund cannot execute twice
- ✓ Wallet cannot become negative
- ✓ Client cannot manipulate final booking price
- ✓ ₹10,000 discount limit cannot be bypassed
- ✓ Cancellation cannot bypass the 24-hour rule
- ✓ Admin adjustments create audit records
- ✓ Historical ledger records cannot be silently modified
- ✓ Chat is isolated by Team
- ✓ Privileged credentials never reach the client

These checks are explicitly listed in the source blueprint's production security validation section.

---

# 9.31 AI-Assisted Development Security Rule

Because this MVP may be implemented using an AI coding agent, security rules must be supplied to the agent explicitly.

The agent must **not infer security solely from the UI**.

For every sensitive function, the implementation specification should identify:

Function

↓

Authenticated?

↓

Required role?

↓

Team association?

↓

Current state?

↓

Business validation?

↓

Atomic transaction?

↓

Ledger / audit required?

↓

Idempotency required?

For example:

`confirm_booking()`

Required:

- authenticated user
- Host / Co-host / authorized Admin
- valid Team relationship
- valid hold
- slot still available
- active membership
- sufficient Team credits
- valid participants
- backend-calculated final price
- atomic wallet + ledger + booking update
- duplicate confirmation protection

This prevents an AI coding agent from creating a function that technically works but violates the financial and authorization model.

---

## 9.32 Final Security Principle

The core rule for the entire Thrill Mill Club MVP is:

NEVER TRUST THE CLIENT  
WITH:

Money  
Credits  
Inventory  
Permissions  
Historical Truth

The mobile application can request:

"Book this slot."  
"Request membership."  
"Cancel this booking."  
"Add this participant."

But the backend must decide:

"Is this user allowed?"  
"Is this Team correct?"  
"Is the price correct?"  
"Are the credits available?"  
"Is the Turf slot available?"  
"Has this already happened?"  
"Does the cancellation qualify?"  
"Should the transaction be committed?"

Then:

DATABASE



Records authoritative result

This is the final security principle stated in the source blueprint and should guide the implementation of every sensitive feature.